PRACTICAL 6

PART A — QUESTION 1

Task: Remote control interfaces: create a Switchable interface with a default toggle() method and implement it in Fan and Light classes. Demonstrate switching each device from a main program.

PART A — QUESTION 2

Task: Notification senders: create a Notifier functional interface, send messages using Email and SMS implementations, add an Urgent marker interface, and demonstrate both an anonymous class and a lambda expression.

PART B — PROJECT WORK: MINIBANK

Task: Extend the MiniBank project with Transactable and InterestBearing interfaces, a WithdrawRule functional interface, a Premium marker interface, package-based classes and a runnable application.

SOURCE CODE — PART A QUESTION 1

All code below is set in Consolas, 9 pt. These files implement Part A Question 1.

1/Main.java

```
interface Switchable
{
    void on();
    void off();

    default void toggle()
    {
        on();
    }
}

@FunctionalInterface
interface PowerRule
{
    boolean canTurnOn(Switchable device, int hour);
}

class Fan implements Switchable
{
    public void on()
    {
        System.out.println("Fan ON");
    }

    public void off()
    {
        System.out.println("Fan OFF");
    }
}

class Light implements Switchable
{
    public void on()
    {
        System.out.println("Light ON");
    }

    public void off()
    {
        System.out.println("Light OFF");
```

```
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Switchable[] devices = {new Fan(), new Light()};

        for (Switchable s : devices)
        {
            s.toggle();
        }

        PowerRule nightRule = new PowerRule()
        {
            public boolean canTurnOn(Switchable device, int hour)
            {
                return hour >= 18 && hour <= 23;
            }
        };

        PowerRule dayRule = (device, hour) -> hour >= 6 && hour < 18;

        System.out.println("\n\nNight rule (20h): "
                + nightRule.canTurnOn(devices[0], 20));

        System.out.println("Day rule (20h): "
                + dayRule.canTurnOn(devices[1], 20));
    }
}
```

```
Fan ON
Light ON


Night rule (20h): true
Day rule (20h): false
```

SOURCE CODE — PART A QUESTION 2

All code below is set in Consolas, 9 pt. These files implement Part A Question 2.

2/NotificationDemo.java

```
public class NotificationDemo
{
      public static void main(String[] args)
      {
            Notifier email=(message)->
                  System.out.println("Email: "+message);

            Notifier sms=(message)->
                  System.out.println("SMS: "+message);

            Notifier[] senders=
            {
                  email,
                  sms
            };

            String message="Your payment was successful.";

            System.out.println("----- NORMAL BROADCAST -----");
```

```
                for(Notifier sender:senders)
                {
                        sender.send(message);
                }

                System.out.println("\n----- URGENT SENDERS -----");

                Notifier urgentEmail=new EmailNotifier();

                Notifier[] allSenders=
                {
                        urgentEmail,
                        sms
                };

                for(Notifier sender:allSenders)
                {
                        sender.send(message);

                        if(sender instanceof Urgent)
                        {
                                sender.send(message);
                        }
                }
        }
}
```

2/EmailNotifier.java

```
public class EmailNotifier implements Notifier, Urgent
{
    public void send(String message)
    {
        System.out.println("Email: " + message);
    }
}
```

2/Notifier.java

```
@FunctionalInterface
public interface Notifier
{
    void send(String message);
}
```

2/SMSNotifier.java

```
public class SMSNotifier implements Notifier
{
    public void send(String message)
    {
        System.out.println("SMS: " + message);
    }
}
```

2/Urgent.java

```
public interface Urgent
{
}
```

```
----- NORMAL BROADCAST -----
Email: Your payment was successful.
SMS: Your payment was successful.

----- URGENT SENDERS -----
Email: Your payment was successful.
Email: Your payment was successful.
SMS: Your payment was successful.
```

SOURCE CODE — PART B MINIBANK PROJECT

All code below is set in Consolas, 9 pt. MiniBank source files are arranged by their project folder structure.

MiniBank.java

```
import model.Account;
import model.CurrentAccount;
import model.FixedDepositAccount;
import model.Premium;
import model.SavingsAccount;
import model.WithdrawRule;
import service.AccountService;

import static util.Validator.isValidMobile;

public class MiniBank {

    public static void main(String[] args) {

        System.out.println("========== MiniBank - Practical 6 ==========");

        Account savings = new SavingsAccount("Riya", 5000, 1000);
        Account current = new CurrentAccount("Aarav", 2000, 5000);
        Account fixedDeposit = new FixedDepositAccount("Meera", 10000);

        Account[] accounts = {savings, current, fixedDeposit};

        System.out.println("\n--- Account Details ---");
        for (Account account : accounts) {
            AccountService.printAccount(account);
        }

        System.out.println("--- WithdrawRule: Anonymous Class ---");

        WithdrawRule anonymousRule = new WithdrawRule() {
            @Override
            public boolean allow(Account account, long amount) {
                return amount > 0 && amount <= account.getBalance();
            }
        };

        System.out.println(
                "Anonymous rule withdrawal: "
                        + AccountService.withdraw(savings, 1000, anonymousRule)
        );

        System.out.println("\n--- WithdrawRule: Lambda ---");

        WithdrawRule lambdaRule =
                (account, amount) -> amount > 0 && amount <= 2000;

        System.out.println(
                "Lambda rule withdrawal: "
                        + AccountService.withdraw(current, 1500, lambdaRule)
```

```
        );

        System.out.println("\n--- Premium Marker Interface ---");

        Premium premiumCustomer = new Premium() {
        };

        if (premiumCustomer instanceof Premium) {
            System.out.println("Premium object detected.");
        }

        System.out.println("\n--- Static Import Example ---");
        System.out.println("9876543210 valid mobile: " + isValidMobile("9876543210"));
        System.out.println("12345 valid mobile: " + isValidMobile("12345"));

        System.out.println("\n--- Final Balances ---");
        for (Account account : accounts) {
            System.out.println(account);
        }

        System.out.println("\nMiniBank finished.");
    }
}
```

model/Account.java

```
package model;

public abstract class Account implements Transactable, InterestBearing {
    private static int counter = 1;

    private final String accountNumber;
    private String ownerName;
    protected long balance;
    private boolean active;

    public Account(String ownerName, long openingBalance) {
        this.accountNumber = String.format("AC%04d", counter++);
        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }

    public Account(String ownerName) {
        this(ownerName, 0);
    }

    @Override
    public synchronized void deposit(long amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    @Override
    public synchronized boolean withdraw(long amount) {
        if (amount <= 0 || !active || !canWithdraw(amount)) {
            return false;
        }

        balance -= amount;
        return true;
    }

    public abstract boolean canWithdraw(long amount);
```

```
    @Override
    public long balanceForInterest() {
        return balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }

    public void deactivate() {
        active = false;
    }

    @Override
    public String toString() {
        return accountNumber + " | " + ownerName + " | Balance: " + balance;
    }
}
```

model/BankInfo.java

```
package model;

public record BankInfo(String name, String branch) {
}
```

model/Command.java

```
package model;

public record Command(
        TransactionType type,
        String accountNumber,
        long amount
) {
}
```

model/CurrentAccount.java

```
package model;

public class CurrentAccount extends Account {
    private final long overdraftLimit;

    public CurrentAccount(String ownerName, long openingBalance, long overdraftLimit) {
        super(ownerName, openingBalance);
        this.overdraftLimit = overdraftLimit;
    }

    public CurrentAccount(String ownerName, long openingBalance) {
        this(ownerName, openingBalance, 5000);
    }

    @Override
```

```
    public double interestRate() {
        return 0.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return getBalance() - amount >= -overdraftLimit;
    }

    public long getOverdraftLimit() {
        return overdraftLimit;
    }
}
```

model/Customer.java

```
package model;

public class Customer {
    private final String name;
    private final String email;
    private final String mobile;

    public Customer(String name, String email, String mobile) {
        this.name = name;
        this.email = email;
        this.mobile = mobile;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getMobile() {
        return mobile;
    }
}
```

model/FixedDepositAccount.java

```
package model;

public class FixedDepositAccount extends Account {

    public FixedDepositAccount(String ownerName, long openingBalance) {
        super(ownerName, openingBalance);
    }

    @Override
    public double interestRate() {
        return 7.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return false;
    }
}
```

model/InterestBearing.java

```
package model;

public interface InterestBearing {
    double interestRate();

    default double yearlyInterest() {
        return balanceForInterest() * interestRate() / 100.0;
    }

    // Implementing classes provide the current balance used by the default method.
    long balanceForInterest();
}
```

model/Premium.java

```
package model;

// Marker interface: it has no methods.
public interface Premium {
}
```

model/SavingsAccount.java

```
package model;

public class SavingsAccount extends Account {
    private final long minBalance;

    public SavingsAccount(String ownerName, long openingBalance, long minBalance) {
        super(ownerName, openingBalance);
        this.minBalance = minBalance;
    }

    public SavingsAccount(String ownerName, long openingBalance) {
        this(ownerName, openingBalance, 1000);
    }

    @Override
    public double interestRate() {
        return 4.0;
    }

    @Override
    public boolean canWithdraw(long amount) {
        return getBalance() - amount >= minBalance;
    }

    public long getMinBalance() {
        return minBalance;
    }
}
```

model/Transactable.java

```
package model;

public interface Transactable {
    void deposit(long amount);
    boolean withdraw(long amount);
}
```

model/TransactionType.java

```
package model;
```

```
public enum TransactionType {
    DEPOSIT,
    WITHDRAW,
    TRANSFER
}
```

model/WithdrawRule.java

```
package model;

@FunctionalInterface
public interface WithdrawRule {
    boolean allow(Account account, long amount);
}
```

service/AccountService.java

```
package service;

import model.Account;
import model.WithdrawRule;

public class AccountService {

    public static boolean withdraw(Account account, long amount, WithdrawRule rule) {
        if (rule.allow(account, amount)) {
            return account.withdraw(amount);
        }
        return false;
    }

    public static void printAccount(Account account) {
        System.out.println(account);
        System.out.println("Interest rate: " + account.interestRate() + "%");
        System.out.println("Yearly interest: " + account.yearlyInterest());
        System.out.println();
    }
}
```

service/CommandParser.java

```
package service;

import model.Command;
import model.TransactionType;

public final class CommandParser {

    private CommandParser() {
    }

    public static Command parse(String line) {
        if (line == null) {
            throw new IllegalArgumentException("Command cannot be null");
        }

        String[] parts = line.trim().split("\\s+");

        if (parts.length != 3) {
            throw new IllegalArgumentException(
                    "Command must contain: TYPE ACCOUNT_NUMBER AMOUNT");
        }

        TransactionType type =
                TransactionType.valueOf(parts[0].toUpperCase());
```

```
        long amount;
        try {
            amount = Long.parseLong(parts[2]);
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("Invalid amount");
        }

        return new Command(type, parts[1], amount);
    }
}
```

service/StatementFormatter.java

```
package service;

import model.Account;

public final class StatementFormatter {

    private StatementFormatter() {
    }

    public static String buildStatement(Account account) {
        StringBuilder sb = new StringBuilder();

        sb.append("----- MiniBank Statement -----\n");
        sb.append("Account Number: ")
                .append(account.getAccountNumber()).append('\n');
        sb.append("Owner: ")
                .append(account.getOwnerName()).append('\n');
        sb.append("Balance: ")
                .append(account.getBalance()).append('\n');
        sb.append("Interest Rate: ")
                .append(account.interestRate()).append("%\n");
        sb.append("Active: ")
                .append(account.isActive()).append('\n');

        return sb.toString();
    }
}
```

util/Validator.java

```
package util;

import java.util.regex.Pattern;

public class Validator {
    private static final Pattern MOBILE =
            Pattern.compile("^[6-9][0-9]{9}$");

    private static final Pattern EMAIL =
            Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");

    public static boolean isValidMobile(String mobile) {
        return mobile != null && MOBILE.matcher(mobile).matches();
    }

    public static boolean isValidEmail(String email) {
        return email != null && EMAIL.matcher(email).matches();
    }
}
```

```
========== MiniBank - Practical 6 ==========

--- Account Details ---
AC0001 | Riya | Balance: 5000
Interest rate: 4.0%
Yearly interest: 200.0

AC0002 | Aarav | Balance: 2000
Interest rate: 0.0%
Yearly interest: 0.0

AC0003 | Meera | Balance: 10000
Interest rate: 7.0%
Yearly interest: 700.0

--- WithdrawRule: Anonymous Class ---
Anonymous rule withdrawal: true

--- WithdrawRule: Lambda ---
Lambda rule withdrawal: true

--- Premium Marker Interface ---
Premium object detected.

--- Static Import Example ---
9876543210 valid mobile: true
12345 valid mobile: false

--- Final Balances ---
AC0001 | Riya | Balance: 4000
AC0002 | Aarav | Balance: 500
AC0003 | Meera | Balance: 10000

MiniBank finished.
```

KEY QUESTIONS AND ANSWERS

Q1. What is the difference between an interface and an abstract class?

Answer: An interface specifies a contract that classes can implement, while an abstract class can provide shared state, constructors and partial implementation. A class may implement several interfaces but extend only one class.

Q2. What is a functional interface, and why are lambda expressions useful with it?

Answer: A functional interface has one abstract method. A lambda expression supplies that method in short, readable form, which is useful for small behaviours such as withdrawal rules or notifications.

Q3. Why are default methods useful in interfaces?

Answer: A default method lets an interface add shared behaviour without forcing every existing implementing class to be changed. For example, toggle() can call an implementing class's switch method.

PRACTICAL 7

PART A — QUESTION 1

Task: Form validator: define custom validation annotations, annotate a form model, and use reflection to inspect fields and print validation results.

PART A — QUESTION 2

Task: Mini test runner: define a custom @Run annotation and use reflection to locate and execute each method marked with @Run.

PART B — PROJECT WORK: MINIBANK

Task: Add @Id, @Positive and @MaxLength annotations to the MiniBank model and build an AnnotationValidator that inspects fields through reflection and reports invalid account data.

SOURCE CODE — PART A QUESTION 1

All code below is set in Consolas, 9 pt. These files implement Part A Question 1.

FormValidatorDemo.java

```
import java.lang.annotation.*;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}

class SignupForm {
    @NotBlank
    @MaxLength(20)
    String username;

    @NotBlank
    @MaxLength(50)
    String email;

    @MaxLength(10)
    String phone;

    SignupForm(String username, String email, String phone) {
        this.username = username;
        this.email = email;
        this.phone = phone;
    }
}

class FormChecker {
    public static List<String> validate(Object object) {
        List<String> errors = new ArrayList<>();

        for (Field field : object.getClass().getDeclaredFields()) {
            try {
```

```
                field.setAccessible(true);
                Object value = field.get(object);

                if (field.isAnnotationPresent(NotBlank.class)) {
                    if (value == null ||
                        value.toString().trim().isEmpty()) {
                        errors.add(field.getName() + " must not be blank");
                    }
                }

                if (field.isAnnotationPresent(MaxLength.class)) {
                    int max = field.getAnnotation(MaxLength.class).value();

                    if (value != null &&
                        value.toString().length() > max) {
                        errors.add(field.getName() +
                                " must have maximum length " + max);
                    }
                }
            } catch (IllegalAccessException e) {
                errors.add("Cannot access field: " + field.getName());
            }
        }

        return errors;
    }
}

public class FormValidatorDemo {
    public static void main(String[] args) {
        SignupForm form =
                new SignupForm(
                        "",
                        "verylongemailaddress@example.com",
                        "12345678901"
                );

        List<String> errors = FormChecker.validate(form);

        if (errors.isEmpty()) {
            System.out.println("Form is valid");
        } else {
            for (String error : errors) {
                System.out.println(error);
            }
        }
    }
}
```

```
username must not be blank
phone must have maximum length 10
```

SOURCE CODE — PART A QUESTION 2

All code below is set in Consolas, 9 pt. These files implement Part A Question 2.

MiniTestRunner.java

```
import java.lang.annotation.*;
import java.lang.reflect.Method;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
```

```
@interface Run {
}

class MyTests {

    @Run
    public void testAddition() {
        System.out.println("testAddition executed");
    }

    @Run
    public void testSubtraction() {
        System.out.println("testSubtraction executed");
    }

    public void normalMethod() {
        System.out.println("normalMethod executed");
    }

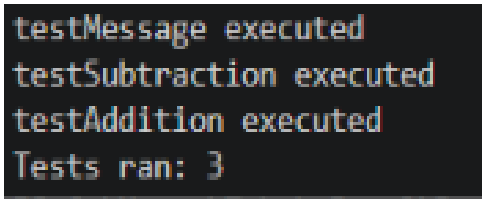
    @Run
    public void testMessage() {
        System.out.println("testMessage executed");
    }
}

public class MiniTestRunner {

    public static void main(String[] args) {
        MyTests tests = new MyTests();
        int count = 0;

        for (Method method : MyTests.class.getDeclaredMethods()) {
            if (method.isAnnotationPresent(Run.class)
                    && method.getParameterCount() == 0) {
                try {
                    method.invoke(tests);
                    count++;
                } catch (Exception e) {
                    System.out.println(
                            "Failed: " + method.getName());
                }
            }
        }

        System.out.println("Tests ran: " + count);
    }
}
```



SOURCE CODE — PART B MINIBANK PROJECT

All code below is set in Consolas, 9 pt. MiniBank source files are arranged by their project folder structure.

MiniBank.java

```
import java.util.Scanner;
import model.Account;
import model.BankInfo;
import model.Command;
```

```
import model.CurrentAccount;
import model.Customer;
import model.FixedDepositAccount;
import model.MenuOption;
import model.SavingsAccount;
import model.WithdrawRule;
import service.BankService;
import util.AnnotationValidator;
import util.CommandParser;
import util.Validator;

public class MiniBank
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        BankInfo bankInfo = new BankInfo("MiniBank", "Charusat Branch");
        System.out.println("================================");
        System.out.println(bankInfo);
        System.out.println("================================");

        Customer customer = new Customer(
                "Riya",
                "riya@example.com",
                "9876543210"
        );
        customer.setAddress(
                new Customer.Address("University Road", "Anand", "388001")
        );

        Account savings = new SavingsAccount("Riya", 5000, 1000);
        Account current = new CurrentAccount("Rahul", 3000, 2000);
        Account fixed = new FixedDepositAccount("Amit", 10000);

        Account[] accounts = { savings, current, fixed };

        System.out.println("\n--- Accounts / Polymorphism ---");
        for (Account account : accounts)
        {
            System.out.println(account);
            System.out.println("Interest Rate: " + account.interestRate() + "%");
            System.out.println("Yearly Interest: " + account.yearlyInterest());

            if (account instanceof FixedDepositAccount)
            {
                System.out.println("Type: Fixed Deposit (locked)");
            }
        }

        System.out.println("\n--- Deposit / Withdraw ---");
        savings.deposit(2000);
        System.out.println("Savings balance after deposit: " + savings.getBalance());

        System.out.println("Withdraw 3000: " + savings.withdraw(3000));
        System.out.println("Savings balance: " + savings.getBalance());

        System.out.println("Withdraw 10000: " + savings.withdraw(10000));
        System.out.println("Savings balance: " + savings.getBalance());

        System.out.println("\n--- Interfaces ---");

        WithdrawRule anonymousRule = new WithdrawRule()
        {
            @Override
            public boolean allow(Account account, long amount)
```

```
        {
            return account.canWithdraw(amount);
        }
    };

    WithdrawRule lambdaRule =
            (account, amount) -> account.canWithdraw(amount);

    System.out.println("Anonymous WithdrawRule: "
            + anonymousRule.allow(savings, 500));

    System.out.println("Lambda WithdrawRule: "
            + lambdaRule.allow(savings, 500));

    System.out.println("Premium account? "
            + (savings instanceof model.Premium));

    System.out.println("\n--- Validation");
    System.out.println("Valid mobile: "
            + Validator.isValidMobile("9876543210"));
    System.out.println("Invalid mobile: "
            + Validator.isValidMobile("12345"));
    System.out.println("Valid email: "
            + Validator.isValidEmail("riya@example.com"));
    System.out.println("Invalid email: "
            + Validator.isValidEmail("wrong-email"));

    System.out.println("\n--- Command Parser ---");
    Command command = CommandParser.parse("DEPOSIT AC0001 500");
    if (command != null)
    {
        System.out.println("Type: " + command.type());
        System.out.println("Account: " + command.accountNumber());
        System.out.println("Amount: " + command.amount());
    }

    System.out.println("\n--- Reflection");
    Account invalidAccount = new SavingsAccount("Test", -100, 0);
    String[] errors = AnnotationValidator.validate(invalidAccount);

    if (errors.length == 0)
    {
        System.out.println("No validation errors");
    }
    else
    {
        for (String error : errors)
        {
            System.out.println(error);
        }
    }

    System.out.println("\n--- Valid Account Validation ---");
    String[] validErrors = AnnotationValidator.validate(savings);

    if (validErrors.length == 0)
    {
        System.out.println("No validation errors");
    }
    else
    {
        for (String error : validErrors)
        {
            System.out.println(error);
        }
    }
```

```
        System.out.println("\n--- Statement ---");
        System.out.println(BankService.buildStatement(savings));

        System.out.println("\n--- Customer ---");
        System.out.println("Customer ID: " + customer.getCustomerId());
        System.out.println("Name: " + customer.getName());
        System.out.println("Address: " + customer.getAddress());

        Customer clonedCustomer = customer.clone();
        System.out.println("Clone created: "
                + (clonedCustomer != null));

        System.out.println("\n--- Menu Shell");
        runMenu(sc);

        sc.close();
    }

    private static void runMenu(Scanner sc)
    {
        boolean running = true;


    }
}
```

model/Account.java

```
package model;

import model.annotation.Id;
import model.annotation.MaxLength;
import model.annotation.Positive;

public abstract class Account
        implements Transactable, InterestBearing
{
    @Id
    private final String accountNumber;

    @MaxLength(30)
    private String ownerName;

    @Positive
    protected long balance;

    private boolean active;

    private static int accountCounter = 1;

    private static String generateAccountNumber()
    {
        return String.format("AC%04d", accountCounter++);
    }

    public Account(String ownerName, long openingBalance)
    {
        this.accountNumber = generateAccountNumber();
        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }

    public Account(String ownerName)
    {
```

```
        this(ownerName, 0);
    }

    public String getAccountNumber()
    {
        return accountNumber;
    }

    public String getOwnerName()
    {
        return ownerName;
    }

    public long getBalance()
    {
        return balance;
    }

    public boolean isActive()
    {
        return active;
    }

    @Override
    public void deposit(long amount)
    {
        if (amount > 0)
        {
            balance += amount;
        }
    }

    @Override
    public boolean withdraw(long amount)
    {
        if (amount > 0 && canWithdraw(amount))
        {
            balance -= amount;
            return true;
        }

        return false;
    }

    public abstract double interestRate();

    public abstract boolean canWithdraw(long amount);

    @Override
    public double balanceForInterest()
    {
        return balance;
    }

    @Override
    public String toString()
    {
        return accountNumber + " | "
                + ownerName + " | "
                + balance + " | active=" + active;
    }

    @Override
    public boolean equals(Object obj)
    {
        if (this == obj)
```

```
        {
            return true;
        }

        if (!(obj instanceof Account))
        {
            return false;
        }

        Account other = (Account) obj;

        return accountNumber.equals(other.accountNumber);
    }

    @Override
    public int hashCode()
    {
        return accountNumber.hashCode();
    }
}
```

model/annotation/Id.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Id
{
}
```

model/annotation/MaxLength.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface MaxLength
{
    int value();
}
```

model/annotation/Positive.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Positive
{
    String message() default "must be > 0";
```

```
}
```

model/BankInfo.java

```
package model;

public record BankInfo(String name, String branch)
{
}
```

model/Command.java

```
package model;

public record Command(
        TransactionType type,
        String accountNumber,
        long amount)
{
}
```

model/CurrentAccount.java

```
package model;

public class CurrentAccount extends Account
{
    private long overdraftLimit;

    public CurrentAccount(
            String ownerName,
            long balance,
            long overdraftLimit)
    {
        super(ownerName, balance);
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public double interestRate()
    {
        return 0.0;
    }

    @Override
    public boolean canWithdraw(long amount)
    {
        return amount >= 0
                && balance - amount >= -overdraftLimit;
    }
}
```

model/Customer.java

```
package model;

public class Customer implements Cloneable
{
    private String name;
    private String email;
    private String mobile;
    private final String customerId;

    private static long customerCounter = 100;

    private Address address;
```

```
    private static String generateCustomerId()
    {
        return "CUST" + (++customerCounter);
    }

    public Customer(String name, String email, String mobile)
    {
        this.customerId = generateCustomerId();
        this.name = name;
        this.email = email;
        this.mobile = mobile;
    }

    public String getName()
    {
        return name;
    }

    public String getEmail()
    {
        return email;
    }

    public String getMobile()
    {
        return mobile;
    }

    public String getCustomerId()
    {
        return customerId;
    }

    public Address getAddress()
    {
        return address;
    }

    public void setAddress(Address address)
    {
        this.address = address;
    }

    @Override
    public Customer clone()
    {
        try
        {
            Customer copy = (Customer) super.clone();

            if (address != null)
            {
                copy.address = new Address(
                        address.getLine(),
                        address.getCity(),
                        address.getPincode()
                );
            }

            return copy;
        }
        catch (CloneNotSupportedException e)
        {
            return null;
        }
```

```
    }

    public static class Address
    {
        private String line;
        private String city;
        private String pincode;

        public Address(String line, String city, String pincode)
        {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }

        public String getLine()
        {
            return line;
        }

        public String getCity()
        {
            return city;
        }

        public String getPincode()
        {
            return pincode;
        }

        @Override
        public String toString()
        {
            return line + ", " + city + " - " + pincode;
        }
    }
}
```

model/FixedDepositAccount.java

```
package model;

public class FixedDepositAccount extends Account
{
    public FixedDepositAccount(String ownerName, long balance)
    {
        super(ownerName, balance);
    }

    @Override
    public double interestRate()
    {
        return 7.0;
    }

    @Override
    public boolean canWithdraw(long amount)
    {
        return false;
    }
}
```

model/InterestBearing.java

```
package model;
```

```
public interface InterestBearing
{
    default double yearlyInterest()
    {
        return balanceForInterest() * interestRate() / 100.0;
    }

    double interestRate();

    double balanceForInterest();
}
```

model/MenuOption.java

```
package model;

public enum MenuOption
{
    OPEN_ACCOUNT,
    DEPOSIT,
    WITHDRAW,
    TRANSFER,
    EXIT
}
```

model/Premium.java

```
package model;

public interface Premium
{
}
```

model/SavingsAccount.java

```
package model;

public class SavingsAccount extends Account implements Premium
{
    private long minBalance;

    public SavingsAccount(String ownerName, long balance, long minBalance)
    {
        super(ownerName, balance);
        this.minBalance = minBalance;
    }

    public SavingsAccount(String ownerName)
    {
        this(ownerName, 0, 1000);
    }

    @Override
    public double interestRate()
    {
        return 4.0;
    }

    @Override
    public boolean canWithdraw(long amount)
    {
        return amount >= 0 && balance - amount >= minBalance;
    }
}
```

model/Transactable.java

```
package model;

public interface Transactable
{
    void deposit(long amount);

    boolean withdraw(long amount);
}
```

model/TransactionType.java

```
package model;

public enum TransactionType
{
    DEPOSIT,
    WITHDRAW,
    TRANSFER
}
```

model/WithdrawRule.java

```
package model;

@FunctionalInterface
public interface WithdrawRule
{
    boolean allow(Account account, long amount);
}
```

service/BankService.java

```
package service;

import model.Account;
import util.StatementFormatter;

public class BankService
{
    public static boolean transfer(
            Account from,
            Account to,
            long amount)
    {
        if (from.withdraw(amount))
        {
            to.deposit(amount);
            return true;
        }

        return false;
    }

    public static String buildStatement(Account account)
    {
        return StatementFormatter.buildStatement(account);
    }
}
```

util/AnnotationValidator.java

```
package util;

import model.annotation.MaxLength;
import model.annotation.Positive;
```

```
import java.lang.reflect.Field;
import java.util.ArrayList;

public class AnnotationValidator
{
    public static String[] validate(Object obj)
    {
        ArrayList<String> errors = new ArrayList<>();

        if (obj == null)
        {
            errors.add("Object cannot be null");
            return errors.toArray(new String[0]);
        }

        Field[] fields = obj.getClass().getDeclaredFields();

        for (Field field : fields)
        {
            field.setAccessible(true);

            try
            {
                if (field.isAnnotationPresent(Positive.class))
                {
                    Positive positive =
                            field.getAnnotation(Positive.class);

                    Object value = field.get(obj);

                    if (value instanceof Number)
                    {
                        long number =
                                ((Number) value).longValue();

                        if (number <= 0)
                        {
                            errors.add(positive.message());
                        }
                    }
                }

                if (field.isAnnotationPresent(MaxLength.class))
                {
                    MaxLength maxLength =
                            field.getAnnotation(MaxLength.class);

                    Object value = field.get(obj);

                    if (value instanceof String)
                    {
                        String text = (String) value;

                        if (text.length() > maxLength.value())
                        {
                            errors.add(
                                    field.getName()
                                    + " length must be <= "
                                    + maxLength.value()
                            );
                        }
                    }
                }
            }
            catch (IllegalAccessException e)
```

```
            {
                errors.add("Cannot access field: "
                        + field.getName());
            }
        }

        return errors.toArray(new String[0]);
    }
}
```

util/CommandParser.java

```
package util;

import model.Command;
import model.TransactionType;

public class CommandParser
{
    public static Command parse(String line)
    {
        if (line == null)
        {
            return null;
        }

        String[] parts = line.trim().split("\\s+");

        if (parts.length != 3)
        {
            return null;
        }

        try
        {
            TransactionType type =
                    TransactionType.valueOf(parts[0].toUpperCase());

            long amount = Long.parseLong(parts[2]);

            if (amount <= 0)
            {
                return null;
            }

            return new Command(type, parts[1], amount);
        }
        catch (IllegalArgumentException e)
        {
            return null;
        }
    }
}
```

util/StatementFormatter.java

```
package util;

import model.Account;

public class StatementFormatter
{
    public static String buildStatement(Account account)
    {
        StringBuilder statement = new StringBuilder();
```

```
        statement.append("----- ACCOUNT STATEMENT -----\n");
        statement.append("Account Number: ")
                .append(account.getAccountNumber())
                .append("\n");
        statement.append("Owner: ")
                .append(account.getOwnerName())
                .append("\n");
        statement.append("Balance: ")
                .append(account.getBalance())
                .append("\n");
        statement.append("Active: ")
                .append(account.isActive())
                .append("\n");
        statement.append("----------------------------");

        return statement.toString();
    }
}
```

util/Validator.java

```
package util;

import java.util.regex.Pattern;

public class Validator
{
    private static final Pattern MOBILE =
            Pattern.compile("^[6-9][0-9]{9}$");

    private static final Pattern EMAIL =
            Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");

    private static final Pattern PAN =
            Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");

    private static final Pattern IFSC =
            Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

    public static boolean isValidMobile(String value)
    {
        return value != null && MOBILE.matcher(value).matches();
    }

    public static boolean isValidEmail(String value)
    {
        return value != null && EMAIL.matcher(value).matches();
    }

    public static boolean isValidPan(String value)
    {
        return value != null && PAN.matcher(value).matches();
    }

    public static boolean isValidIfsc(String value)
    {
        return value != null && IFSC.matcher(value).matches();
    }

    public static boolean isValidAmount(String value)
    {
        return value != null && value.matches("^[1-9][0-9]*$");
    }
}
```

```
==============================
BankInfo[name=MiniBank, branch=Charusat Branch]
==============================

--- Accounts / Polymorphism ---
AC0001 | Riya | 5000 | active=true
Interest Rate: 4.0%
Yearly Interest: 200.0
AC0002 | Rahul | 3000 | active=true
Interest Rate: 0.0%
Yearly Interest: 0.0
AC0003 | Amit | 10000 | active=true
Interest Rate: 7.0%
Yearly Interest: 700.0
Type: Fixed Deposit (locked)

--- Deposit / Withdraw ---
Savings balance after deposit: 7000
Withdraw 3000: true
Savings balance: 4000
Withdraw 10000: false
Savings balance: 4000

--- Interfaces ---
Anonymous WithdrawRule: true
Lambda WithdrawRule: true
Premium account? true

--- Validation
Valid mobile: true
Invalid mobile: false
Valid email: true
Invalid email: false

--- Command Parser ---
Type: DEPOSIT
Account: AC0001
Amount: 500

--- Reflection
No validation errors
```

```
--- Reflection
No validation errors

--- Valid Account Validation ---
No validation errors

--- Statement ---
----- ACCOUNT STATEMENT -----
Account Number: AC0001
Owner: Riya
Balance: 4000
Active: true
-----------------------------

--- Customer ---
Customer ID: CUST101
Name: Riya
Address: University Road, Anand - 388001
Clone created: true
```

KEY QUESTIONS AND ANSWERS

Q1. What do @Retention and @Target control?

Answer: @Retention controls how long an annotation is kept, such as at runtime for reflection. @Target specifies where it may be used, for example on fields, methods or types.

Q2. How does reflection use getDeclaredFields()?

Answer: getDeclaredFields() returns the fields declared in a class. A validator can inspect each field, read its annotations and check the current field value against the stated rule.

Q3. How can annotations and reflection validate generic objects?

Answer: The validator can iterate over an object's fields at runtime and apply rules based on annotations. The same validation logic can therefore work with different annotated model classes.

PRACTICAL 8

PART A — QUESTION 1

Task: Guarded calculator: accept numeric input, handle invalid values and division by zero with a custom exception, and use finally to display a completion message.

PART A — QUESTION 2

Task: Stock issue: validate issue quantities and throw custom checked exceptions for invalid quantity and insufficient stock, reporting the available quantity and shortfall.

PART B — PROJECT WORK: MINIBANK

Task: Create a MiniBank exception hierarchy for invalid amounts, missing accounts and insufficient funds; handle transfer failures safely and use try-with-resources for data access.

SOURCE CODE — PART A QUESTION 1

All code below is set in Consolas, 9 pt. These files implement Part A Question 1.

1/GuardedCalculator.java

```
import java.util.Scanner;

class DivideByZeroException extends Exception
{
    public DivideByZeroException(String message)
    {
        super(message);
    }
}

public class GuardedCalculator
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        boolean success = false;
        int attempt = 0;

        while (!success)
        {
            attempt++;

            try
            {
                System.out.println("\n===== Calculator Attempt " + attempt + " =====");

                System.out.print("Enter first number: ");
                double num1 = Double.parseDouble(sc.nextLine());

                System.out.print("Enter second number: ");
                double num2 = Double.parseDouble(sc.nextLine());

                System.out.print("Enter operator (+, -, *, /): ");
                char operator = sc.nextLine().charAt(0);

                double result;

                switch (operator)
                {
                    case '+':
```

```
                        result = num1 + num2;
                        break;

                    case '-':
                        result = num1 - num2;
                        break;

                    case '*':
                        result = num1 * num2;
                        break;

                    case '/':
                        if (num2 == 0)
                        {
                            throw new DivideByZeroException(
                                "Cannot divide by zero."
                            );
                        }

                        result = num1 / num2;
                        break;

                    default:
                        throw new IllegalArgumentException(
                            "Invalid operator. Use +, -, * or /."
                        );
                }

                System.out.println("Result = " + result);

                success = true;
            }
            catch (NumberFormatException e)
            {
                System.out.println(
                    "Invalid number! Please enter valid numeric values."
                );
            }
            catch (DivideByZeroException e)
            {
                System.out.println("Error: " + e.getMessage());
            }
            catch (IllegalArgumentException e)
            {
                System.out.println("Error: " + e.getMessage());
            }
            finally
            {
                System.out.println("Attempt " + attempt + " completed.");
            }
        }

        System.out.println("\nCalculation completed successfully!");

        sc.close();
    }
}
```

```
===== Calculator Attempt 1 =====
Enter first number: 10
Enter second number: 5
Enter operator (+, -, *, /): %
Error: Invalid operator. Use +, -, * or /.
Attempt 1 completed.

===== Calculator Attempt 2 =====
Enter first number: 10
Enter second number: 2
Enter operator (+, -, *, /): /
Result = 5.0
Attempt 2 completed.

Calculation completed successfully!
```

SOURCE CODE — PART A QUESTION 2

All code below is set in Consolas, 9 pt. These files implement Part A Question 2.

2/InvalidQuantityException.java

```
public class InvalidQuantityException extends Exception {

    public InvalidQuantityException(String message) {
        super(message);
    }
}
```

2/OutOfStockException.java

```
public class OutOfStockException extends Exception {
    private int shortfall;

    public OutOfStockException(int shortfall) {
        super("Out of stock. Shortfall: " + shortfall);
        this.shortfall = shortfall;
    }

    public int getShortfall() {
        return shortfall;
    }
}
```

2/Request.java

```
public class Request {

    String item;
    int quantity;

    public Request(String item, int quantity) {
        this.item = item;
        this.quantity = quantity;
    }
}
```

2/StockIssue.java

```
import java.util.Arrays;
import java.util.List;

public class StockIssue {

    public static void main(String[] args) {
```

```
        Warehouse warehouse = new Warehouse();

        // Initial warehouse stock
        warehouse.addStock("Laptop", 10);
        warehouse.addStock("Mouse", 20);
        warehouse.addStock("Keyboard", 15);

        // List of issue requests
        List<Request> requests = Arrays.asList(
                new Request("Laptop", 5),
                new Request("Mouse", 25),
                new Request("Keyboard", 0),
                new Request("Monitor", 3),
                new Request("Keyboard", 10),
                new Request("Laptop", -2),
                new Request("Mouse", 5)
        );

        System.out.println("===== PROCESSING STOCK REQUESTS =====");

        // Process every request
        for (Request request : requests) {

            System.out.println(
                    "\nRequest: " +
                    request.quantity + " " +
                    request.item
            );

            try {
                warehouse.issue(
                        request.item,
                        request.quantity
                );
            }
            catch (OutOfStockException e) {
                System.out.println(
                        "FAILED: Not enough " + request.item +
                        ". Shortfall = " +
                        e.getShortfall()
                );
            }
            catch (InvalidQuantityException e) {
                System.out.println(
                        "FAILED: " + e.getMessage()
                );
            }
        }

        warehouse.displayStock();
    }
}
```

2/Warehouse.java

```
import java.util.HashMap;
import java.util.Map;

public class Warehouse {

    private Map<String, Integer> stock = new HashMap<>();

    public void addStock(String item, int qty) {
        stock.put(item, stock.getOrDefault(item, 0) + qty);
    }
```

```
    public void issue(String item, int qty)
            throws OutOfStockException, InvalidQuantityException {

        if (qty <= 0) {
            throw new InvalidQuantityException(
                    "Invalid quantity: " + qty +
                    ". Quantity must be greater than 0."
            );
        }

        int available = stock.getOrDefault(item, 0);

        if (qty > available) {
            int shortfall = qty - available;
            throw new OutOfStockException(shortfall);
        }

        stock.put(item, available - qty);

        System.out.println(
                "SUCCESS: Issued " + qty + " " + item +
                ". Remaining stock: " + (available - qty)
        );
    }

    public void displayStock() {
        System.out.println("\nCurrent Warehouse Stock:");

        for (Map.Entry<String, Integer> entry : stock.entrySet()) {
            System.out.println(
                    entry.getKey() + " : " + entry.getValue()
            );
        }
    }
}
```

```
===== PROCESSING STOCK REQUESTS =====

Request: 5 Laptop
SUCCESS: Issued 5 Laptop. Remaining stock: 5

Request: 25 Mouse
FAILED: Not enough Mouse. Shortfall = 5

Request: 0 Keyboard
FAILED: Invalid quantity: 0. Quantity must be greater than 0.

Request: 3 Monitor
FAILED: Not enough Monitor. Shortfall = 3

Request: 10 Keyboard
SUCCESS: Issued 10 Keyboard. Remaining stock: 5

Request: -2 Laptop
FAILED: Invalid quantity: -2. Quantity must be greater than 0.

Request: 5 Mouse
SUCCESS: Issued 5 Mouse. Remaining stock: 15

Current Warehouse Stock:
Laptop : 5
Mouse : 15
Keyboard : 5
```

SOURCE CODE — PART B MINIBANK PROJECT

All code below is set in Consolas, 9 pt. MiniBank source files are arranged by their project folder structure.

MiniBank.java

```
import exception.AccountNotFoundException;
import exception.BankException;
import exception.InsufficientFundsException;
import exception.InvalidAmountException;
import java.util.Scanner;
import model.Account;
import model.BankInfo;
import model.Command;
import model.CurrentAccount;
import model.Customer;
import model.FixedDepositAccount;
import model.SavingsAccount;
import model.WithdrawRule;
import service.BankService;
import util.AnnotationValidator;
import util.CommandParser;
import util.Validator;

public class MiniBank
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        BankInfo bankInfo =
                new BankInfo("MiniBank", "Charusat Branch");

        System.out.println("================================");
        System.out.println(bankInfo);
        System.out.println("MINIBANK");
        System.out.println("================================");

        Customer customer = new Customer(
                "Riya",
                "riya@example.com",
                "9876543210"
        );

        customer.setAddress(
                new Customer.Address(
                        "University Road",
                        "Anand",
                        "388001"
                )
        );

        Account savings =
                new SavingsAccount("Riya", 5000, 1000);

        Account current =
                new CurrentAccount("Rahul", 3000, 2000);

        Account fixed =
                new FixedDepositAccount("Amit", 10000);

        Account[] accounts = { savings, current, fixed };

        System.out.println("\n--- Accounts / Polymorphism ---");

        for (Account account : accounts)
```

```
        {
            System.out.println(account);
            System.out.println(
                    "Interest Rate: "
                    + account.interestRate() + "%");

            System.out.println(
                    "Yearly Interest: "
                    + account.yearlyInterest());

            if (account instanceof FixedDepositAccount)
            {
                System.out.println(
                        "Type: Fixed Deposit (locked)");
            }
        }

        System.out.println("\n--- Exception Handling ---");

        try
        {
            savings.deposit(2000);

            System.out.println(
                    "Savings balance after deposit: "
                    + savings.getBalance());

            savings.withdraw(3000);

            System.out.println(
                    "Savings balance after withdrawal: "
                    + savings.getBalance());
        }
        catch (InvalidAmountException e)
        {
            System.out.println(
                    "Invalid amount: " + e.getMessage());
        }
        catch (InsufficientFundsException e)
        {
            System.out.println(
                    "Insufficient funds: " + e.getMessage());
            System.out.println(
                    "Shortfall: " + e.getShortfall());
        }
        finally
        {
            System.out.println(
                    "Deposit/withdraw operation completed.");
        }

        try
        {
            savings.withdraw(0);
        }
        catch (BankException e)
        {
            System.out.println(
                    "Handled invalid withdrawal: "
                    + e.getMessage());
        }
        finally
        {
            System.out.println(
                    "Invalid-withdrawal attempt completed.");
        }
```

```
        // Insufficient funds example.
        try
        {
            savings.withdraw(10000);
        }
        catch (InsufficientFundsException e)
        {
            System.out.println(
                    "Handled insufficient funds: "
                    + e.getMessage());
            System.out.println(
                    "Shortfall: " + e.getShortfall());
        }
        catch (InvalidAmountException e)
        {
            System.out.println(
                    "Handled invalid amount: "
                    + e.getMessage());
        }
        finally
        {
            System.out.println(
                    "Large-withdrawal attempt completed.");
        }

        System.out.println("\n--- Transfer ---");

        try
        {
            BankService.transfer(
                    savings,
                    current,
                    1000
            );
        }
        catch (BankException e)
        {
            System.out.println(
                    "Transfer could not be completed: "
                    + e.getMessage());
        }
        finally
        {
            System.out.println("Transfer request completed.");
        }

        // Account-not-found example.
        try
        {
            Account missingAccount = findAccount(
                    accounts, "AC9999");

            BankService.transfer(
                    savings,
                    missingAccount,
                    500
            );
        }
        catch (AccountNotFoundException e)
        {
            System.out.println(
                    "Account error: " + e.getMessage());
        }
        catch (BankException e)
        {
```

```
            System.out.println(
                    "Bank error: " + e.getMessage());
        }
        finally
        {
            System.out.println(
                    "Account search/transfer attempt completed.");
        }



        WithdrawRule anonymousRule = new WithdrawRule()
        {
            @Override
            public boolean allow(Account account, long amount)
            {
                return account.canWithdraw(amount);
            }
        };

        WithdrawRule lambdaRule =
                (account, amount) ->
                        account.canWithdraw(amount);

        System.out.println(
                "Anonymous WithdrawRule: "
                + anonymousRule.allow(savings, 500));

        System.out.println(
                "Lambda WithdrawRule: "
                + lambdaRule.allow(savings, 500));

        System.out.println(
                "Premium account? "
                + (savings instanceof model.Premium));

        System.out.println("\n--- Validation ---");

        System.out.println(
                "Valid mobile: "
                + Validator.isValidMobile("9876543210"));

        System.out.println(
                "Invalid mobile: "
                + Validator.isValidMobile("12345"));

        System.out.println(
                "Valid email: "
                + Validator.isValidEmail(
                        "riya@example.com"));

        System.out.println(
                "Invalid email: "
                + Validator.isValidEmail(
                        "wrong-email"));

        System.out.println("\n--- Command Parser ---");

        Command command =
                CommandParser.parse(
                        "DEPOSIT AC0001 500");

        if (command != null)
        {
            System.out.println(
                    "Type: " + command.type());
```

```
            System.out.println(
                    "Account: " + command.accountNumber());
            System.out.println(
                    "Amount: " + command.amount());
        }

        System.out.println("\n--- Reflection Validation ---");

        Account invalidAccount =
                new SavingsAccount("Test", -100, 0);

        String[] errors =
                AnnotationValidator.validate(
                        invalidAccount);

        for (String error : errors)
        {
            System.out.println(error);
        }

        System.out.println("\n--- Statement ---");
        System.out.println(
                BankService.buildStatement(savings));

        System.out.println("\n--- Customer ---");
        System.out.println(
                "Customer ID: " + customer.getCustomerId());
        System.out.println(
                "Name: " + customer.getName());
        System.out.println(
                "Address: " + customer.getAddress());

        Customer clonedCustomer =
                customer.clone();

        System.out.println(
                "Clone created: "
                + (clonedCustomer != null));


        sc.close();
    }

    private static Account findAccount(
            Account[] accounts,
            String accountNumber)
            throws AccountNotFoundException
    {
        for (Account account : accounts)
        {
            if (account.getAccountNumber()
                    .equals(accountNumber))
            {
                return account;
            }
        }

        throw new AccountNotFoundException(
                "Account " + accountNumber
                + " was not found.");
    }
}
```

exception/AccountNotFoundException.java

```
package exception;

public class AccountNotFoundException extends BankException
{
    public AccountNotFoundException(String message)
    {
        super(message);
    }
}
```

exception/BankException.java

```
package exception;

public class BankException extends Exception
{
    public BankException(String message)
    {
        super(message);
    }
}
```

exception/InsufficientFundsException.java

```
package exception;

public class InsufficientFundsException extends BankException
{
    private final long shortfall;

    public InsufficientFundsException(String message, long shortfall)
    {
        super(message);
        this.shortfall = shortfall;
    }

    public long getShortfall()
    {
        return shortfall;
    }
}
```

exception/InvalidAmountException.java

```
package exception;

public class InvalidAmountException extends BankException
{
    public InvalidAmountException(String message)
    {
        super(message);
    }
}
```

model/Account.java

```
package model;

import exception.BankException;
import exception.InvalidAmountException;
import exception.InsufficientFundsException;
import model.annotation.Id;
import model.annotation.MaxLength;
import model.annotation.Positive;
```

```
public abstract class Account
        implements Transactable, InterestBearing
{
    @Id
    private final String accountNumber;

    @MaxLength(30)
    private String ownerName;

    @Positive
    protected long balance;

    private boolean active;

    private static int accountCounter = 1;

    private static String generateAccountNumber()
    {
        return String.format("AC%04d", accountCounter++);
    }

    public Account(String ownerName, long openingBalance)
    {
        this.accountNumber = generateAccountNumber();
        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }

    public Account(String ownerName)
    {
        this(ownerName, 0);
    }

    public String getAccountNumber()
    {
        return accountNumber;
    }

    public String getOwnerName()
    {
        return ownerName;
    }

    public long getBalance()
    {
        return balance;
    }

    public boolean isActive()
    {
        return active;
    }

    @Override
    public void deposit(long amount) throws InvalidAmountException
    {
        if (amount <= 0)
        {
            throw new InvalidAmountException(
                    "Deposit amount must be greater than zero.");
        }

        balance += amount;
    }
```

```
    @Override
    public boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException
    {
        if (amount <= 0)
        {
            throw new InvalidAmountException(
                    "Withdrawal amount must be greater than zero.");
        }

        if (!canWithdraw(amount))
        {
            long available = Math.max(0, balance);
            long shortfall = amount - available;

            throw new InsufficientFundsException(
                    "Insufficient funds. Balance: " + balance
                    + ", requested: " + amount,
                    Math.max(0, shortfall));
        }

        balance -= amount;
        return true;
    }

    public void transfer(Account to, long amount) throws BankException
    {
        try
        {
            if (to == null)
            {
                throw new exception.AccountNotFoundException(
                        "Destination account was not found.");
            }

            withdraw(amount);
            to.deposit(amount);

            System.out.println(
                    "Transfer successful: " + amount
                    + " from " + getAccountNumber()
                    + " to " + to.getAccountNumber());
        }
        catch (BankException e)
        {
            System.out.println(
                    "Transfer failed: " + e.getMessage());
            throw e;
        }
        finally
        {
            System.out.println(
                    "Transfer operation finished for "
                    + getAccountNumber());
        }
    }

    public abstract double interestRate();

    public abstract boolean canWithdraw(long amount);

    @Override
    public double balanceForInterest()
    {
        return balance;
    }
```

```
    @Override
    public String toString()
    {
        return accountNumber + " | "
                + ownerName + " | "
                + balance + " | active=" + active;
    }

    @Override
    public boolean equals(Object obj)
    {
        if (this == obj)
        {
            return true;
        }

        if (!(obj instanceof Account))
        {
            return false;
        }

        Account other = (Account) obj;

        return accountNumber.equals(other.accountNumber);
    }

    @Override
    public int hashCode()
    {
        return accountNumber.hashCode();
    }
}
```

model/annotation/Id.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Id
{
}
```

model/annotation/MaxLength.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface MaxLength
{
    int value();
}
```

model/annotation/Positive.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Positive
{
    String message() default "must be > 0";
}
```

model/BankInfo.java

```
package model;

public record BankInfo(String name, String branch)
{
}
```

model/Command.java

```
package model;

public record Command(
        TransactionType type,
        String accountNumber,
        long amount)
{
}
```

model/CurrentAccount.java

```
package model;

public class CurrentAccount extends Account
{
    private long overdraftLimit;

    public CurrentAccount(
            String ownerName,
            long balance,
            long overdraftLimit)
    {
        super(ownerName, balance);
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public double interestRate()
    {
        return 0.0;
    }

    @Override
    public boolean canWithdraw(long amount)
    {
        return amount >= 0
                && balance - amount >= -overdraftLimit;
    }
}
```

model/Customer.java

```
package model;

public class Customer implements Cloneable
{
    private String name;
    private String email;
    private String mobile;
    private final String customerId;

    private static long customerCounter = 100;

    private Address address;

    private static String generateCustomerId()
    {
        return "CUST" + (++customerCounter);
    }

    public Customer(String name, String email, String mobile)
    {
        this.customerId = generateCustomerId();
        this.name = name;
        this.email = email;
        this.mobile = mobile;
    }

    public String getName()
    {
        return name;
    }

    public String getEmail()
    {
        return email;
    }

    public String getMobile()
    {
        return mobile;
    }

    public String getCustomerId()
    {
        return customerId;
    }

    public Address getAddress()
    {
        return address;
    }

    public void setAddress(Address address)
    {
        this.address = address;
    }

    @Override
    public Customer clone()
    {
        try
        {
            Customer copy = (Customer) super.clone();

            if (address != null)
```

```
            {
                copy.address = new Address(
                        address.getLine(),
                        address.getCity(),
                        address.getPincode()
                );
            }

            return copy;
        }
        catch (CloneNotSupportedException e)
        {
            return null;
        }
    }

    public static class Address
    {
        private String line;
        private String city;
        private String pincode;

        public Address(String line, String city, String pincode)
        {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }

        public String getLine()
        {
            return line;
        }

        public String getCity()
        {
            return city;
        }

        public String getPincode()
        {
            return pincode;
        }

        @Override
        public String toString()
        {
            return line + ", " + city + " - " + pincode;
        }
    }
}
```

model/FixedDepositAccount.java

```
package model;

public class FixedDepositAccount extends Account
{
    public FixedDepositAccount(String ownerName, long balance)
    {
        super(ownerName, balance);
    }

    @Override
    public double interestRate()
    {
```

```
            return 7.0;
        }

        @Override
        public boolean canWithdraw(long amount)
        {
            return false;
        }
    }
```

model/InterestBearing.java

```
    package model;

    public interface InterestBearing
    {
        default double yearlyInterest()
        {
            return balanceForInterest() * interestRate() / 100.0;
        }

        double interestRate();

        double balanceForInterest();
    }
```

model/MenuOption.java

```
    package model;

    public enum MenuOption
    {
        OPEN_ACCOUNT,
        DEPOSIT,
        WITHDRAW,
        TRANSFER,
        EXIT
    }
```

model/Premium.java

```
    package model;

    public interface Premium
    {
    }
```

model/SavingsAccount.java

```
    package model;

    public class SavingsAccount extends Account implements Premium
    {
        private long minBalance;

        public SavingsAccount(String ownerName, long balance, long minBalance)
        {
            super(ownerName, balance);
            this.minBalance = minBalance;
        }

        public SavingsAccount(String ownerName)
        {
            this(ownerName, 0, 1000);
        }
```

```
    @Override
    public double interestRate()
    {
        return 4.0;
    }

    @Override
    public boolean canWithdraw(long amount)
    {
        return amount >= 0 && balance - amount >= minBalance;
    }
}
```

model/Transactable.java

```
package model;

import exception.InvalidAmountException;
import exception.InsufficientFundsException;

public interface Transactable
{
    void deposit(long amount) throws InvalidAmountException;

    boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException;
}
```

model/TransactionType.java

```
package model;

public enum TransactionType
{
    DEPOSIT,
    WITHDRAW,
    TRANSFER
}
```

model/WithdrawRule.java

```
package model;

@FunctionalInterface
public interface WithdrawRule
{
    boolean allow(Account account, long amount);
}
```

service/BankService.java

```
package service;

import exception.BankException;
import model.Account;
import util.StatementFormatter;

public class BankService
{
    public static boolean transfer(
            Account from,
            Account to,
            long amount) throws BankException
    {
        from.transfer(to, amount);
        return true;
```

```
    }

    public static String buildStatement(Account account)
    {
        return StatementFormatter.buildStatement(account);
    }
}
```

util/AnnotationValidator.java

```
package util;

import model.annotation.MaxLength;
import model.annotation.Positive;

import java.lang.reflect.Field;
import java.util.ArrayList;

public class AnnotationValidator
{
    public static String[] validate(Object obj)
    {
        ArrayList<String> errors = new ArrayList<>();

        if (obj == null)
        {
            errors.add("Object cannot be null");
            return errors.toArray(new String[0]);
        }

        Field[] fields = obj.getClass().getDeclaredFields();

        for (Field field : fields)
        {
            field.setAccessible(true);

            try
            {
                if (field.isAnnotationPresent(Positive.class))
                {
                    Positive positive =
                            field.getAnnotation(Positive.class);

                    Object value = field.get(obj);

                    if (value instanceof Number)
                    {
                        long number =
                                ((Number) value).longValue();

                        if (number <= 0)
                        {
                            errors.add(positive.message());
                        }
                    }
                }

                if (field.isAnnotationPresent(MaxLength.class))
                {
                    MaxLength maxLength =
                            field.getAnnotation(MaxLength.class);

                    Object value = field.get(obj);

                    if (value instanceof String)
                    {
```

```
                        String text = (String) value;

                        if (text.length() > maxLength.value())
                        {
                            errors.add(
                                    field.getName()
                                    + " length must be <= "
                                    + maxLength.value()
                            );
                        }
                    }
                }
            }
            catch (IllegalAccessException e)
            {
                errors.add("Cannot access field: "
                        + field.getName());
            }
        }

        return errors.toArray(new String[0]);
    }
}
```

util/CommandParser.java

```
package util;

import model.Command;
import model.TransactionType;

public class CommandParser
{
    public static Command parse(String line)
    {
        if (line == null)
        {
            return null;
        }

        String[] parts = line.trim().split("\\s+");

        if (parts.length != 3)
        {
            return null;
        }

        try
        {
            TransactionType type =
                    TransactionType.valueOf(parts[0].toUpperCase());

            long amount = Long.parseLong(parts[2]);

            if (amount <= 0)
            {
                return null;
            }

            return new Command(type, parts[1], amount);
        }
        catch (IllegalArgumentException e)
        {
            return null;
        }
    }
```

```
}
```

util/StatementFormatter.java

```
package util;

import model.Account;

public class StatementFormatter
{
    public static String buildStatement(Account account)
    {
        StringBuilder statement = new StringBuilder();

        statement.append("----- ACCOUNT STATEMENT -----\n");
        statement.append("Account Number: ")
                .append(account.getAccountNumber())
                .append("\n");
        statement.append("Owner: ")
                .append(account.getOwnerName())
                .append("\n");
        statement.append("Balance: ")
                .append(account.getBalance())
                .append("\n");
        statement.append("Active: ")
                .append(account.isActive())
                .append("\n");
        statement.append("-----------------------------");

        return statement.toString();
    }
}
```

util/Validator.java

```
package util;

import java.util.regex.Pattern;

public class Validator
{
    private static final Pattern MOBILE =
            Pattern.compile("^[6-9][0-9]{9}$");

    private static final Pattern EMAIL =
            Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");

    private static final Pattern PAN =
            Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");

    private static final Pattern IFSC =
            Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

    public static boolean isValidMobile(String value)
    {
        return value != null && MOBILE.matcher(value).matches();
    }

    public static boolean isValidEmail(String value)
    {
        return value != null && EMAIL.matcher(value).matches();
    }

    public static boolean isValidPan(String value)
    {
        return value != null && PAN.matcher(value).matches();
```

```
    }

    public static boolean isValidIfsc(String value)
    {
        return value != null && IFSC.matcher(value).matches();
    }

    public static boolean isValidAmount(String value)
    {
        return value != null && value.matches("^[1-9][0-9]*$");
    }
}
```

```
==============================
BankInfo[name=MiniBank, branch=Charusat Branch]
MINIBANK
==============================

--- Accounts / Polymorphism ---
AC0001 | Riya | 5000 | active=true
Interest Rate: 4.0%
Yearly Interest: 200.0
AC0002 | Rahul | 3000 | active=true
Interest Rate: 0.0%
Yearly Interest: 0.0
AC0003 | Amit | 10000 | active=true
Interest Rate: 7.0%
Yearly Interest: 700.0
Type: Fixed Deposit (locked)

--- Exception Handling ---
Savings balance after deposit: 7000
Savings balance after withdrawal: 4000
Deposit/withdraw operation completed.
Handled invalid withdrawal: Withdrawal amount must be greater than zero.
Invalid-withdrawal attempt completed.
Handled insufficient funds: Insufficient funds. Balance: 4000, requested: 10000
Shortfall: 6000
Large-withdrawal attempt completed.

--- Transfer ---
Transfer successful: 1000 from AC0001 to AC0002
Transfer operation finished for AC0001
Transfer request completed.
Account error: Account AC9999 was not found.
Account search/transfer attempt completed.
Anonymous WithdrawRule: true
Lambda WithdrawRule: true
Premium account? true

--- Validation ---
Valid mobile: true
Invalid mobile: false
Valid email: true
```

```
--- Validation ---
Valid mobile: true
Invalid mobile: false
Valid email: true
Invalid email: false

--- Command Parser ---
Type: DEPOSIT
Account: AC0001
Amount: 500

--- Reflection Validation ---

--- Statement ---
----- ACCOUNT STATEMENT -----
Account Number: AC0001
Owner: Riya
Balance: 3000
Active: true
-----------------------------

--- Customer ---
Customer ID: CUST101
Name: Riya
Address: University Road, Anand - 388001
Clone created: true
```

KEY QUESTIONS AND ANSWERS

Q1. What is the difference between checked and unchecked exceptions?

Answer: Checked exceptions must be handled or declared by the caller and are used for recoverable conditions. Unchecked exceptions extend RuntimeException and usually represent programming errors or invalid use.

Q2. What is the purpose of a finally block?

Answer: A finally block runs after try and catch processing whether an exception occurs or not. It is useful for cleanup and for printing a final status message.

Q3. How does try-with-resources guarantee closing resources?

Answer: Resources declared in the try header are closed automatically when the block ends, including when an exception is thrown. This prevents resource leaks and makes cleanup reliable.

PRACTICAL 9

PART A — QUESTION 1

Task: Counter race: run several threads that update one shared counter, observe the race condition, and correct the update using synchronized access.

PART A — QUESTION 2

Task: Seat booking race: allow multiple booking threads to request the same limited seat count, then synchronize the booking operation to prevent overbooking.

PART B — PROJECT WORK: MINIBANK

Task: Create MiniBank AccountWorker Runnable tasks that perform concurrent deposits, start and join several threads, protect Account updates with synchronization and display thread names.

SOURCE CODE — PART A QUESTION 1

All code below is set in Consolas, 9 pt. These files implement Part A Question 1.

1/CounterRace.java

```
public class CounterRace
{
    private static int counter = 0;

    private static final int THREAD_COUNT = 10;
    private static final int INCREMENTS_PER_THREAD = 100_000;

    public static void main(String[] args)
            throws InterruptedException
    {
        Thread[] threads = new Thread[THREAD_COUNT];

        System.out.println("=== Practical 9: Counter Race ===");
        System.out.println("Threads: " + THREAD_COUNT);
        System.out.println(
                "Increments per thread: " + INCREMENTS_PER_THREAD);

        // Unsynchronized run: demonstrates the race condition.
        counter = 0;

        for (int i = 0; i < THREAD_COUNT; i++)
        {
            threads[i] = new Thread(() ->
            {
                for (int j = 0; j < INCREMENTS_PER_THREAD; j++)
                {
                    counter++;
                }
            });

            threads[i].start();
        }

        for (Thread thread : threads)
        {
            thread.join();
        }

        int expected =
                THREAD_COUNT * INCREMENTS_PER_THREAD;
```

```
        System.out.println("\n--- Without Synchronization ---");
        System.out.println("Expected counter: " + expected);
        System.out.println("Actual counter:   " + counter);
        System.out.println(
                "Race condition occurred: " + (counter != expected));

        // Synchronized run: every increment is protected.
        counter = 0;

        for (int i = 0; i < THREAD_COUNT; i++)
        {
            threads[i] = new Thread(() ->
            {
                for (int j = 0; j < INCREMENTS_PER_THREAD; j++)
                {
                    incrementSynchronized();
                }
            });

            threads[i].start();
        }

        for (Thread thread : threads)
        {
            thread.join();
        }

        System.out.println("\n--- With Synchronization ---");
        System.out.println("Expected counter: " + expected);
        System.out.println("Actual counter:   " + counter);
        System.out.println(
                "Exactly right: " + (counter == expected));
    }

    private static synchronized void incrementSynchronized()
    {
        counter++;
    }
}
```

```
=== Practical 9: Counter Race ===
Threads: 10
Increments per thread: 100000

--- Without Synchronization ---
Expected counter: 1000000
Actual counter:   172257
Race condition occurred: true

--- With Synchronization ---
Expected counter: 1000000
Actual counter:   1000000
Exactly right: true
```

SOURCE CODE — PART A QUESTION 2

All code below is set in Consolas, 9 pt. These files implement Part A Question 2.

2/SeatBookingRace.java

```
import java.util.concurrent.atomic.AtomicInteger;

public class SeatBookingRace
```

```
 {
     private static final int TOTAL_SEATS = 5;
     private static final int THREAD_COUNT = 10;

     public static void main(String[] args)
             throws InterruptedException
     {
         System.out.println("=== Practical 9: Seat Booking Race ===");
         System.out.println("Available seats: " + TOTAL_SEATS);
         System.out.println("Booking threads: " + THREAD_COUNT);

         runWithoutSynchronization();
         runWithSynchronization();
     }

     private static void runWithoutSynchronization()
             throws InterruptedException
     {
         UnsafeBooking booking =
                 new UnsafeBooking(TOTAL_SEATS);

         Thread[] threads = new Thread[THREAD_COUNT];

         for (int i = 0; i < THREAD_COUNT; i++)
         {
             final int customerId = i + 1;

             threads[i] = new Thread(() ->
             {
                 if (booking.bookSeat())
                 {
                     System.out.println(
                             "Customer " + customerId
                             + " booked a seat.");
                 }
             });
         }

         for (Thread thread : threads)
         {
             thread.start();
         }

         for (Thread thread : threads)
         {
             thread.join();
         }

         System.out.println("\n--- Without Synchronization ---");
         System.out.println(
                 "Successful bookings: "
                 + booking.getSuccessfulBookings());
         System.out.println(
                 "Expected maximum: " + TOTAL_SEATS);

         if (booking.getSuccessfulBookings() > TOTAL_SEATS)
         {
             System.out.println(
                     "Oversell occurred: MORE THAN 5 bookings succeeded.");
         }
         else
         {
             System.out.println(
                     "This run did not visibly oversell, but the code "
                     + "is still unsafe.");
         }
```

```
    }

    private static void runWithSynchronization()
            throws InterruptedException
    {
        SafeBooking booking =
                new SafeBooking(TOTAL_SEATS);

        Thread[] threads = new Thread[THREAD_COUNT];

        for (int i = 0; i < THREAD_COUNT; i++)
        {
            final int customerId = i + 1;

            threads[i] = new Thread(() ->
            {
                if (booking.bookSeat())
                {
                    System.out.println(
                            "Customer " + customerId
                            + " booked a seat.");
                }
            });
        }

        for (Thread thread : threads)
        {
            thread.start();
        }

        for (Thread thread : threads)
        {
            thread.join();
        }

        System.out.println("\n--- With Synchronization ---");
        System.out.println(
                "Successful bookings: "
                + booking.getSuccessfulBookings());
        System.out.println(
                "Expected: exactly " + TOTAL_SEATS);
        System.out.println(
                "Exactly 5 succeeded: "
                + (booking.getSuccessfulBookings() == TOTAL_SEATS));
    }

    /*
     * Deliberately unsafe version.
     * The small pause between checking and booking makes the
     * check-then-act race easier to observe.
     */
    private static class UnsafeBooking
    {
        private final int totalSeats;
        private int successfulBookings = 0;

        UnsafeBooking(int totalSeats)
        {
            this.totalSeats = totalSeats;
        }

        boolean bookSeat()
        {
            if (successfulBookings < totalSeats)
            {
                try
```

```
                {
                    Thread.sleep(10);
                }
                catch (InterruptedException e)
                {
                    Thread.currentThread().interrupt();
                }

                successfulBookings++;
                return true;
            }

            return false;
        }

        int getSuccessfulBookings()
        {
            return successfulBookings;
        }
    }

    /*
     * Synchronized version.
     * The complete check-and-update operation is atomic with
     * respect to the other booking threads.
     */
    private static class SafeBooking
    {
        private final AtomicInteger successfulBookings =
                new AtomicInteger(0);

        private final int totalSeats;

        SafeBooking(int totalSeats)
        {
            this.totalSeats = totalSeats;
        }

        synchronized boolean bookSeat()
        {
            if (successfulBookings.get() >= totalSeats)
            {
                return false;
            }

            successfulBookings.incrementAndGet();
            return true;
        }

        int getSuccessfulBookings()
        {
            return successfulBookings.get();
        }
    }
}
```

```
=== Practical 9: Seat Booking Race ===
Available seats: 5
Booking threads: 10
Customer 5 booked a seat.
Customer 2 booked a seat.
Customer 9 booked a seat.
Customer 3 booked a seat.
Customer 4 booked a seat.
Customer 1 booked a seat.
Customer 10 booked a seat.
Customer 8 booked a seat.
Customer 6 booked a seat.
Customer 7 booked a seat.

--- Without Synchronization ---
Successful bookings: 5
Expected maximum: 5
This run did not visibly oversell, but the code is still unsafe.
Customer 4 booked a seat.
Customer 1 booked a seat.
Customer 2 booked a seat.
Customer 3 booked a seat.
Customer 9 booked a seat.

--- With Synchronization ---
Successful bookings: 5
Expected: exactly 5
Exactly 5 succeeded: true
```

SOURCE CODE — PART B MINIBANK PROJECT

All code below is set in Consolas, 9 pt. MiniBank source files are arranged by their project folder structure.

MiniBank.java

```
import exception.BankException;
import model.Account;
import model.RaceAccount;
import model.SavingsAccount;

public class MiniBank
{
    private static final int THREAD_COUNT = 10;
    private static final int TIMES = 1_000;
    private static final long AMOUNT = 10;

    public static void main(String[] args)
            throws InterruptedException
    {
        long expected =
                (long) THREAD_COUNT * TIMES * AMOUNT;

        System.out.println("================================");
        System.out.println("PRACTICAL 9 - PART B: MINIBANK");
        System.out.println("================================");
        System.out.println("Threads: " + THREAD_COUNT);
        System.out.println("Deposits per thread: " + TIMES);
        System.out.println("Amount per deposit: " + AMOUNT);
        System.out.println("Expected balance: " + expected);

        // RUN 1: unsafe account to demonstrate race condition.
        Account unsafeAccount =
                new RaceAccount("Race User", 0);

        System.out.println(
                "\n--- RUN 1: WITHOUT SYNCHRONIZATION ---");
```

```
        runWorkers(unsafeAccount);

        System.out.println(
                "Expected final balance: " + expected);
        System.out.println(
                "Actual final balance:   "
                + unsafeAccount.getBalance());
        System.out.println(
                "Race condition observed: "
                + (unsafeAccount.getBalance() != expected));

        // RUN 2: Practical 8 Account now has synchronized
        // deposit() and withdraw().
        Account safeAccount =
                new SavingsAccount("Safe User", 0, 0);

        System.out.println(
                "\n--- RUN 2: WITH SYNCHRONIZATION ---");

        runWorkers(safeAccount);

        System.out.println(
                "Expected final balance: " + expected);
        System.out.println(
                "Actual final balance:   "
                + safeAccount.getBalance());
        System.out.println(
                "Balance is correct: "
                + (safeAccount.getBalance() == expected));

        // Also verify synchronized withdraw().
        System.out.println(
                "\n--- SYNCHRONIZED WITHDRAW TEST ---");

        try
        {
            safeAccount.withdraw(100);
            System.out.println("Withdrawal successful.");
            System.out.println(
                    "Balance after withdrawal: "
                    + safeAccount.getBalance());
        }
        catch (BankException e)
        {
            System.out.println(
                    "Withdrawal failed: " + e.getMessage());
        }

        System.out.println(
                "\nPractical 9 Part B completed.");
    }

    private static void runWorkers(Account account)
            throws InterruptedException
    {
        Thread[] threads =
                new Thread[THREAD_COUNT];

        for (int i = 0; i < THREAD_COUNT; i++)
        {
            AccountWorker worker =
                    new AccountWorker(
                            account,
                            TIMES,
                            AMOUNT);
```

```
            threads[i] =
                    new Thread(
                            worker,
                            "BankWorker-" + (i + 1));

            System.out.println(
                    "Creating "
                    + threads[i].getName());

            threads[i].start();
        }

        // join() makes main wait for every worker.
        for (Thread thread : threads)
        {
            thread.join();

            System.out.println(
                    thread.getName() + " joined.");
        }
    }
}
```

AccountWorker.java

```
import exception.InvalidAmountException;
import model.Account;

public class AccountWorker implements Runnable
{
    private final Account account;
    private final int times;
    private final long amount;

    public AccountWorker(
            Account account,
            int times,
            long amount)
    {
        this.account = account;
        this.times = times;
        this.amount = amount;
    }

    @Override
    public void run()
    {
        String threadName =
                Thread.currentThread().getName();

        System.out.println(threadName + " started.");

        for (int i = 0; i < times; i++)
        {
            try
            {
                account.deposit(amount);
            }
            catch (InvalidAmountException e)
            {
                System.out.println(
                        threadName
                        + " deposit failed: "
                        + e.getMessage());
            }
```

```
        }

        System.out.println(threadName + " finished.");
    }
}
```

exception/AccountNotFoundException.java

```
package exception;

public class AccountNotFoundException extends BankException
{
    public AccountNotFoundException(String message)
    {
        super(message);
    }
}
```

exception/BankException.java

```
package exception;

public class BankException extends Exception
{
    public BankException(String message)
    {
        super(message);
    }
}
```

exception/InsufficientFundsException.java

```
package exception;

public class InsufficientFundsException extends BankException
{
    private final long shortfall;

    public InsufficientFundsException(String message, long shortfall)
    {
        super(message);
        this.shortfall = shortfall;
    }

    public long getShortfall()
    {
        return shortfall;
    }
}
```

exception/InvalidAmountException.java

```
package exception;

public class InvalidAmountException extends BankException
{
    public InvalidAmountException(String message)
    {
        super(message);
    }
}
```

model/Account.java

```
package model;
```

```
import exception.BankException;
import exception.InvalidAmountException;
import exception.InsufficientFundsException;
import model.annotation.Id;
import model.annotation.MaxLength;
import model.annotation.Positive;

public abstract class Account
        implements Transactable, InterestBearing
{
    @Id
    private final String accountNumber;

    @MaxLength(30)
    private String ownerName;

    @Positive
    protected long balance;

    private boolean active;

    private static int accountCounter = 1;

    private static String generateAccountNumber()
    {
        return String.format("AC%04d", accountCounter++);
    }

    public Account(String ownerName, long openingBalance)
    {
        this.accountNumber = generateAccountNumber();
        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }

    public Account(String ownerName)
    {
        this(ownerName, 0);
    }

    public String getAccountNumber()
    {
        return accountNumber;
    }

    public String getOwnerName()
    {
        return ownerName;
    }

    public long getBalance()
    {
        return balance;
    }

    public boolean isActive()
    {
        return active;
    }

    @Override
    public synchronized void deposit(long amount) throws InvalidAmountException
    {
        if (amount <= 0)
        {
```

```
            throw new InvalidAmountException(
                    "Deposit amount must be greater than zero.");
        }

        balance += amount;
    }

    @Override
    public synchronized boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException
    {
        if (amount <= 0)
        {
            throw new InvalidAmountException(
                    "Withdrawal amount must be greater than zero.");
        }

        if (!canWithdraw(amount))
        {
            long available = Math.max(0, balance);
            long shortfall = amount - available;

            throw new InsufficientFundsException(
                    "Insufficient funds. Balance: " + balance
                    + ", requested: " + amount,
                    Math.max(0, shortfall));
        }

        balance -= amount;
        return true;
    }

    public void transfer(Account to, long amount) throws BankException
    {
        try
        {
            if (to == null)
            {
                throw new exception.AccountNotFoundException(
                        "Destination account was not found.");
            }

            withdraw(amount);
            to.deposit(amount);

            System.out.println(
                    "Transfer successful: " + amount
                    + " from " + getAccountNumber()
                    + " to " + to.getAccountNumber());
        }
        catch (BankException e)
        {
            System.out.println(
                    "Transfer failed: " + e.getMessage());
            throw e;
        }
        finally
        {
            System.out.println(
                    "Transfer operation finished for "
                    + getAccountNumber());
        }
    }

    public abstract double interestRate();
```

```
    public abstract boolean canWithdraw(long amount);

    @Override
    public double balanceForInterest()
    {
        return balance;
    }

    @Override
    public String toString()
    {
        return accountNumber + " | "
                + ownerName + " | "
                + balance + " | active=" + active;
    }

    @Override
    public boolean equals(Object obj)
    {
        if (this == obj)
        {
            return true;
        }

        if (!(obj instanceof Account))
        {
            return false;
        }

        Account other = (Account) obj;

        return accountNumber.equals(other.accountNumber);
    }

    @Override
    public int hashCode()
    {
        return accountNumber.hashCode();
    }
}
```

model/annotation/Id.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Id
{
}
```

model/annotation/MaxLength.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
```

```
@Target(ElementType.FIELD)
public @interface MaxLength
{
    int value();
}
```

model/annotation/Positive.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Positive
{
    String message() default "must be > 0";
}
```

model/BankInfo.java

```
package model;

public record BankInfo(String name, String branch)
{
}
```

model/Command.java

```
package model;

public record Command(
        TransactionType type,
        String accountNumber,
        long amount)
{
}
```

model/CurrentAccount.java

```
package model;

public class CurrentAccount extends Account
{
    private long overdraftLimit;

    public CurrentAccount(
            String ownerName,
            long balance,
            long overdraftLimit)
    {
        super(ownerName, balance);
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public double interestRate()
    {
        return 0.0;
    }

    @Override
    public boolean canWithdraw(long amount)
```

```
    {
        return amount >= 0
                && balance - amount >= -overdraftLimit;
    }
}
```

model/Customer.java

```
package model;

public class Customer implements Cloneable
{
    private String name;
    private String email;
    private String mobile;
    private final String customerId;

    private static long customerCounter = 100;

    private Address address;

    private static String generateCustomerId()
    {
        return "CUST" + (++customerCounter);
    }

    public Customer(String name, String email, String mobile)
    {
        this.customerId = generateCustomerId();
        this.name = name;
        this.email = email;
        this.mobile = mobile;
    }

    public String getName()
    {
        return name;
    }

    public String getEmail()
    {
        return email;
    }

    public String getMobile()
    {
        return mobile;
    }

    public String getCustomerId()
    {
        return customerId;
    }

    public Address getAddress()
    {
        return address;
    }

    public void setAddress(Address address)
    {
        this.address = address;
    }

    @Override
    public Customer clone()
```

```
    {
        try
        {
            Customer copy = (Customer) super.clone();

            if (address != null)
            {
                copy.address = new Address(
                        address.getLine(),
                        address.getCity(),
                        address.getPincode()
                );
            }

            return copy;
        }
        catch (CloneNotSupportedException e)
        {
            return null;
        }
    }

    public static class Address
    {
        private String line;
        private String city;
        private String pincode;

        public Address(String line, String city, String pincode)
        {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }

        public String getLine()
        {
            return line;
        }

        public String getCity()
        {
            return city;
        }

        public String getPincode()
        {
            return pincode;
        }

        @Override
        public String toString()
        {
            return line + ", " + city + " - " + pincode;
        }
    }
}
```

model/FixedDepositAccount.java

```
package model;

public class FixedDepositAccount extends Account
{
    public FixedDepositAccount(String ownerName, long balance)
    {
```

```
        super(ownerName, balance);
    }

    @Override
    public double interestRate()
    {
        return 7.0;
    }

    @Override
    public boolean canWithdraw(long amount)
    {
        return false;
    }
}
```

model/InterestBearing.java

```
package model;

public interface InterestBearing
{
    default double yearlyInterest()
    {
        return balanceForInterest() * interestRate() / 100.0;
    }

    double interestRate();

    double balanceForInterest();
}
```

model/MenuOption.java

```
package model;

public enum MenuOption
{
    OPEN_ACCOUNT,
    DEPOSIT,
    WITHDRAW,
    TRANSFER,
    EXIT
}
```

model/Premium.java

```
package model;

public interface Premium
{
}
```

model/RaceAccount.java

```
package model;

import exception.InsufficientFundsException;
import exception.InvalidAmountException;

public class RaceAccount extends Account
{
    public RaceAccount(String ownerName, long balance)
    {
        super(ownerName, balance);
    }
```

```
    // Deliberately UNSYNCHRONIZED for the first race-condition demo.
    @Override
    public void deposit(long amount)
            throws InvalidAmountException
    {
        if (amount <= 0)
        {
            throw new InvalidAmountException(
                    "Deposit amount must be greater than zero.");
        }

        long oldBalance = balance;
        Thread.yield();
        balance = oldBalance + amount;
    }

    @Override
    public synchronized boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException
    {
        if (amount <= 0)
        {
            throw new InvalidAmountException(
                    "Withdrawal amount must be greater than zero.");
        }

        if (!canWithdraw(amount))
        {
            long shortfall =
                    amount - Math.max(0, balance);

            throw new InsufficientFundsException(
                    "Insufficient funds.",
                    Math.max(0, shortfall));
        }

        balance -= amount;
        return true;
    }

    @Override
    public double interestRate()
    {
        return 0.0;
    }

    @Override
    public boolean canWithdraw(long amount)
    {
        return amount >= 0 && balance >= amount;
    }
}
```

model/SavingsAccount.java

```
package model;

public class SavingsAccount extends Account implements Premium
{
    private long minBalance;

    public SavingsAccount(String ownerName, long balance, long minBalance)
    {
        super(ownerName, balance);
        this.minBalance = minBalance;
```

```
    }

    public SavingsAccount(String ownerName)
    {
        this(ownerName, 0, 1000);
    }

    @Override
    public double interestRate()
    {
        return 4.0;
    }

    @Override
    public boolean canWithdraw(long amount)
    {
        return amount >= 0 && balance - amount >= minBalance;
    }
}
```

model/Transactable.java

```
package model;

import exception.InvalidAmountException;
import exception.InsufficientFundsException;

public interface Transactable
{
    void deposit(long amount) throws InvalidAmountException;

    boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException;
}
```

model/TransactionType.java

```
package model;

public enum TransactionType
{
    DEPOSIT,
    WITHDRAW,
    TRANSFER
}
```

model/WithdrawRule.java

```
package model;

@FunctionalInterface
public interface WithdrawRule
{
    boolean allow(Account account, long amount);
}
```

service/BankService.java

```
package service;

import exception.BankException;
import model.Account;
import util.StatementFormatter;

public class BankService
{
```

```
    public static boolean transfer(
            Account from,
            Account to,
            long amount) throws BankException
    {
        from.transfer(to, amount);
        return true;
    }

    public static String buildStatement(Account account)
    {
        return StatementFormatter.buildStatement(account);
    }
}
```

util/AnnotationValidator.java

```
package util;

import model.annotation.MaxLength;
import model.annotation.Positive;

import java.lang.reflect.Field;
import java.util.ArrayList;

public class AnnotationValidator
{
    public static String[] validate(Object obj)
    {
        ArrayList<String> errors = new ArrayList<>();

        if (obj == null)
        {
            errors.add("Object cannot be null");
            return errors.toArray(new String[0]);
        }

        Field[] fields = obj.getClass().getDeclaredFields();

        for (Field field : fields)
        {
            field.setAccessible(true);

            try
            {
                if (field.isAnnotationPresent(Positive.class))
                {
                    Positive positive =
                            field.getAnnotation(Positive.class);

                    Object value = field.get(obj);

                    if (value instanceof Number)
                    {
                        long number =
                                ((Number) value).longValue();

                        if (number <= 0)
                        {
                            errors.add(positive.message());
                        }
                    }
                }

                if (field.isAnnotationPresent(MaxLength.class))
                {
```

```
                    MaxLength maxLength =
                            field.getAnnotation(MaxLength.class);

                    Object value = field.get(obj);

                    if (value instanceof String)
                    {
                        String text = (String) value;

                        if (text.length() > maxLength.value())
                        {
                            errors.add(
                                    field.getName()
                                    + " length must be <= "
                                    + maxLength.value()
                            );
                        }
                    }
                }
            }
            catch (IllegalAccessException e)
            {
                errors.add("Cannot access field: "
                        + field.getName());
            }
        }

        return errors.toArray(new String[0]);
    }
}
```

util/CommandParser.java

```
package util;

import model.Command;
import model.TransactionType;

public class CommandParser
{
    public static Command parse(String line)
    {
        if (line == null)
        {
            return null;
        }

        String[] parts = line.trim().split("\\s+");

        if (parts.length != 3)
        {
            return null;
        }

        try
        {
            TransactionType type =
                    TransactionType.valueOf(parts[0].toUpperCase());

            long amount = Long.parseLong(parts[2]);

            if (amount <= 0)
            {
                return null;
            }
```

```
            return new Command(type, parts[1], amount);
        }
        catch (IllegalArgumentException e)
        {
            return null;
        }
    }
}
```

util/StatementFormatter.java

```
package util;

import model.Account;

public class StatementFormatter
{
    public static String buildStatement(Account account)
    {
        StringBuilder statement = new StringBuilder();

        statement.append("----- ACCOUNT STATEMENT -----\n");
        statement.append("Account Number: ")
                .append(account.getAccountNumber())
                .append("\n");
        statement.append("Owner: ")
                .append(account.getOwnerName())
                .append("\n");
        statement.append("Balance: ")
                .append(account.getBalance())
                .append("\n");
        statement.append("Active: ")
                .append(account.isActive())
                .append("\n");
        statement.append("-----------------------------");

        return statement.toString();
    }
}
```

util/Validator.java

```
package util;

import java.util.regex.Pattern;

public class Validator
{
    private static final Pattern MOBILE =
            Pattern.compile("^[6-9][0-9]{9}$");

    private static final Pattern EMAIL =
            Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");

    private static final Pattern PAN =
            Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");

    private static final Pattern IFSC =
            Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

    public static boolean isValidMobile(String value)
    {
        return value != null && MOBILE.matcher(value).matches();
    }

    public static boolean isValidEmail(String value)
```

```
    {
        return value != null && EMAIL.matcher(value).matches();
    }

    public static boolean isValidPan(String value)
    {
        return value != null && PAN.matcher(value).matches();
    }

    public static boolean isValidIfsc(String value)
    {
        return value != null && IFSC.matcher(value).matches();
    }

    public static boolean isValidAmount(String value)
    {
        return value != null && value.matches("^[1-9][0-9]*$");
    }
}
```

```
================================
PRACTICAL 9 - PART B: MINIBANK
================================
Threads: 10
Deposits per thread: 1000
Amount per deposit: 10
Expected balance: 100000

--- RUN 1: WITHOUT SYNCHRONIZATION ---
Creating BankWorker-1
Creating BankWorker-2
Creating BankWorker-3
Creating BankWorker-4
Creating BankWorker-5
Creating BankWorker-6
Creating BankWorker-7
Creating BankWorker-8
BankWorker-7 started.
BankWorker-1 started.
BankWorker-8 started.
Creating BankWorker-9
BankWorker-2 started.
BankWorker-4 started.
BankWorker-6 started.
BankWorker-3 started.
BankWorker-5 started.
Creating BankWorker-10
BankWorker-9 started.
BankWorker-10 started.
BankWorker-1 finished.
BankWorker-7 finished.
BankWorker-1 joined.
BankWorker-8 finished.
BankWorker-4 finished.
BankWorker-6 finished.
BankWorker-2 finished.
BankWorker-3 finished.
BankWorker-9 finished.
BankWorker-5 finished.
BankWorker-2 joined.
BankWorker-10 finished.
BankWorker-10 finished.
BankWorker-3 joined.
BankWorker-4 joined.
BankWorker-5 joined.
BankWorker-6 joined.
BankWorker-7 joined.
BankWorker-8 joined.
BankWorker-9 joined.
BankWorker-10 joined.
Expected final balance: 100000
Actual final balance:   19230
Race condition observed: true

--- RUN 2: WITH SYNCHRONIZATION ---
Creating BankWorker-1
Creating BankWorker-2
BankWorker-1 started.
Creating BankWorker-3
BankWorker-1 finished.
BankWorker-2 started.
Creating BankWorker-4
BankWorker-3 started.
BankWorker-2 finished.
Creating BankWorker-5
BankWorker-3 finished.
BankWorker-4 started.
Creating BankWorker-6
BankWorker-4 finished.
BankWorker-5 started.
Creating BankWorker-7
BankWorker-6 started.
Creating BankWorker-8
BankWorker-6 finished.
BankWorker-5 finished.
BankWorker-7 started.
Creating BankWorker-9
BankWorker-8 started.
Creating BankWorker-10
BankWorker-7 finished.
BankWorker-8 finished.
BankWorker-9 started.
```

```
BankWorker-8 finished.
BankWorker-9 started.
BankWorker-1 joined.
BankWorker-2 joined.
BankWorker-3 joined.
BankWorker-10 started.
BankWorker-9 finished.
BankWorker-4 joined.
BankWorker-10 finished.
BankWorker-5 joined.
BankWorker-6 joined.
BankWorker-7 joined.
BankWorker-8 joined.
BankWorker-9 joined.
BankWorker-10 joined.
Expected final balance: 100000
Actual final balance:   100000
Balance is correct: true

--- SYNCHRONIZED WITHDRAW TEST ---
Withdrawal successful.
Balance after withdrawal: 99900

Practical 9 Part B completed.
```

KEY QUESTIONS AND ANSWERS

Q1. What is a race condition?

Answer: A race condition occurs when multiple threads read and modify shared data without proper coordination, so the final result depends on timing and can become incorrect.

Q2. How does synchronized protect shared data?

Answer: synchronized lets only one thread at a time enter the protected method or block for the same lock. This makes each update complete before another thread changes the shared value.

Q3. What are common thread lifecycle states?

Answer: A thread may be new, runnable, blocked, waiting or timed waiting, and terminated. Its state changes as it starts, competes for locks, waits and completes work.

PRACTICAL 10

PART A — QUESTION 1

Task: Thread pool runner: use an ExecutorService with a fixed-size pool to execute multiple named tasks, then shut down the service correctly.

PART A — QUESTION 2

Task: Producer-consumer: coordinate producer and consumer threads over a shared buffer using synchronized methods, wait() and notifyAll().

PART B — PROJECT WORK: MINIBANK

Task: Build a MiniBank transaction processor using an ExecutorService, Runnable transaction tasks, a shared array-buffer producer-consumer example and a transfer strategy that locks accounts in a consistent order.

SOURCE CODE — PART A QUESTION 1

All code below is set in Consolas, 9 pt. These files implement Part A Question 1.

1/ThreadPoolRunner.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class ThreadPoolRunner
{
    private static final int POOL_SIZE = 3;
    private static final int TASK_COUNT = 10;

    public static void main(String[] args)
            throws InterruptedException
    {
        System.out.println("====================================");
        System.out.println("PRACTICAL 10 - THREAD POOL RUNNER");
        System.out.println("====================================");

        ExecutorService executor =
                Executors.newFixedThreadPool(POOL_SIZE);

        List<String> taskNames = new ArrayList<>();

        for (int i = 1; i <= TASK_COUNT; i++)
        {
            final int taskNumber = i;
            String taskName = "Task-" + taskNumber;
            taskNames.add(taskName);

            executor.submit(() ->
            {
                String threadName =
                        Thread.currentThread().getName();

                System.out.println(
                        taskName + " STARTED by "
                        + threadName);

                try
                {
                    Thread.sleep(500);
```

```
                }
                catch (InterruptedException e)
                {
                    Thread.currentThread().interrupt();

                    System.out.println(
                            taskName
                            + " was interrupted by "
                            + threadName);
                    return;
                }

                System.out.println(
                        taskName + " FINISHED by "
                        + threadName);
            });
        }

        System.out.println(
                "\nSubmitted " + TASK_COUNT
                + " tasks to a fixed pool of "
                + POOL_SIZE + " threads.");

        executor.shutdown();

        if (executor.awaitTermination(30, TimeUnit.SECONDS))
        {
            System.out.println(
                    "\nAll submitted tasks completed.");
        }
        else
        {
            System.out.println(
                    "\nTasks did not finish within the time limit.");
            executor.shutdownNow();
        }

        System.out.println(
                "Thread pool runner completed.");
    }
}
```

```
==================================
PRACTICAL 10 - THREAD POOL RUNNER
==================================

Submitted 10 tasks to a fixed pool of 3 threads.
Task-1 STARTED by pool-1-thread-1
Task-2 STARTED by pool-1-thread-2
Task-3 STARTED by pool-1-thread-3
Task-1 FINISHED by pool-1-thread-1
Task-3 FINISHED by pool-1-thread-3
Task-2 FINISHED by pool-1-thread-2
Task-4 STARTED by pool-1-thread-1
Task-5 STARTED by pool-1-thread-3
Task-6 STARTED by pool-1-thread-2
Task-4 FINISHED by pool-1-thread-1
Task-6 FINISHED by pool-1-thread-2
Task-8 STARTED by pool-1-thread-2
Task-5 FINISHED by pool-1-thread-3
Task-7 STARTED by pool-1-thread-1
Task-9 STARTED by pool-1-thread-3
Task-7 FINISHED by pool-1-thread-1
Task-9 FINISHED by pool-1-thread-3
Task-8 FINISHED by pool-1-thread-2
Task-10 STARTED by pool-1-thread-1
Task-10 FINISHED by pool-1-thread-1

All submitted tasks completed.
Thread pool runner completed.
```

SOURCE CODE — PART A QUESTION 2

All code below is set in Consolas, 9 pt. These files implement Part A Question 2.

2/ProducerConsumer.java

```
import java.util.LinkedList;
import java.util.Queue;

public class ProducerConsumer
{
    private static final int BUFFER_SIZE = 4;
    private static final int ITEM_COUNT = 12;

    public static void main(String[] args)
            throws InterruptedException
    {
        System.out.println("====================================");
        System.out.println("PRACTICAL 10 - PRODUCER / CONSUMER");
        System.out.println("====================================");

        SharedBuffer buffer =
                new SharedBuffer(BUFFER_SIZE);

        Thread producer =
                new Thread(
                        new Producer(buffer, ITEM_COUNT),
                        "Producer");

        Thread consumer =
                new Thread(
                        new Consumer(buffer, ITEM_COUNT),
                        "Consumer");

        System.out.println("Starting producer and consumer...");

        producer.start();
```

```
        consumer.start();

        producer.join();
        consumer.join();

        System.out.println(
                "\nBoth threads completed.");
        System.out.println(
                "Produced items: " + buffer.getProducedCount());
        System.out.println(
                "Consumed items: " + buffer.getConsumedCount());
        System.out.println(
                "Items remaining in buffer: "
                + buffer.size());

        boolean correct =
                buffer.getProducedCount() == ITEM_COUNT
                && buffer.getConsumedCount() == ITEM_COUNT
                && buffer.size() == 0;

        System.out.println(
                "No item lost: " + correct);
        System.out.println(
                "No item duplicated: " + correct);
    }

    private static class SharedBuffer
    {
        private final Queue<Integer> buffer =
                new LinkedList<>();

        private final int capacity;

        private int producedCount = 0;
        private int consumedCount = 0;

        SharedBuffer(int capacity)
        {
            this.capacity = capacity;
        }

        public synchronized void put(int item)
                throws InterruptedException
        {
            // Wait while the buffer is full.
            while (buffer.size() == capacity)
            {
                System.out.println(
                        "Producer waiting: buffer is FULL.");
                wait();
            }

            buffer.add(item);
            producedCount++;

            System.out.println(
                    "Producer -> produced "
                    + item
                    + " | buffer = "
                    + buffer);


            notify();
        }

        public synchronized int take()
```

```
            throws InterruptedException
    {

        while (buffer.isEmpty())
        {
            System.out.println(
                    "Consumer waiting: buffer is EMPTY.");
            wait();
        }

        int item = buffer.remove();
        consumedCount++;

        System.out.println(
                "Consumer <- consumed "
                + item
                + " | buffer = "
                + buffer);


        notify();

        return item;
    }

    public synchronized int size()
    {
        return buffer.size();
    }

    public synchronized int getProducedCount()
    {
        return producedCount;
    }

    public synchronized int getConsumedCount()
    {
        return consumedCount;
    }
}


private static class Producer implements Runnable
{
    private final SharedBuffer buffer;
    private final int itemCount;

    Producer(SharedBuffer buffer, int itemCount)
    {
        this.buffer = buffer;
        this.itemCount = itemCount;
    }

    @Override
    public void run()
    {
        try
        {
            for (int i = 1; i <= itemCount; i++)
            {
                buffer.put(i);


                Thread.sleep(100);
            }
        }
```

```
            catch (InterruptedException e)
            {
                Thread.currentThread().interrupt();

                System.out.println(
                        "Producer interrupted.");
            }

            System.out.println("Producer finished.");
        }
    }


    private static class Consumer implements Runnable
    {
        private final SharedBuffer buffer;
        private final int itemCount;

        Consumer(SharedBuffer buffer, int itemCount)
        {
            this.buffer = buffer;
            this.itemCount = itemCount;
        }

        @Override
        public void run()
        {
            try
            {
                for (int i = 1; i <= itemCount; i++)
                {
                    buffer.take();

                    // Small delay to make waiting visible.
                    Thread.sleep(150);
                }
            }
            catch (InterruptedException e)
            {
                Thread.currentThread().interrupt();

                System.out.println(
                        "Consumer interrupted.");
            }

            System.out.println("Consumer finished.");
        }
    }
}
```

```
==================================
PRACTICAL 10 - PRODUCER / CONSUMER
==================================
Starting producer and consumer...
Consumer waiting: buffer is EMPTY.
Producer -> produced 1 | buffer = [1]
Consumer <- consumed 1 | buffer = []
Producer -> produced 2 | buffer = [2]
Consumer <- consumed 2 | buffer = []
Producer -> produced 3 | buffer = [3]
Producer -> produced 4 | buffer = [3, 4]
Consumer <- consumed 3 | buffer = [4]
Producer -> produced 5 | buffer = [4, 5]
Consumer <- consumed 4 | buffer = [5]
Producer -> produced 6 | buffer = [5, 6]
Producer -> produced 7 | buffer = [5, 6, 7]
Consumer <- consumed 5 | buffer = [6, 7]
Producer -> produced 8 | buffer = [6, 7, 8]
Consumer <- consumed 6 | buffer = [7, 8]
Producer -> produced 9 | buffer = [7, 8, 9]
Producer -> produced 10 | buffer = [7, 8, 9, 10]
Consumer <- consumed 7 | buffer = [8, 9, 10]
Producer -> produced 11 | buffer = [8, 9, 10, 11]
Consumer <- consumed 8 | buffer = [9, 10, 11]
Producer -> produced 12 | buffer = [9, 10, 11, 12]
Consumer <- consumed 9 | buffer = [10, 11, 12]
Producer finished.
Consumer <- consumed 10 | buffer = [11, 12]
Consumer <- consumed 11 | buffer = [12]
Consumer <- consumed 12 | buffer = []
Consumer finished.

Both threads completed.
Produced items: 12
Consumer items: 12
Items remaining in buffer: 0
No item lost: true
No item duplicated: true
```

SOURCE CODE — PART B MINIBANK PROJECT

All code below is set in Consolas, 9 pt. MiniBank source files are arranged by their project folder structure.

MiniBankTransactionDemo.java

```
import model.Account;
import model.SavingsAccount;

public class MiniBankTransactionDemo
{
    public static void main(String[] args)
            throws Exception
    {
        Account account =
                new SavingsAccount(
                        "Transaction User",
                        1000,
                        0);

        TransactionProcessor processor =
                new TransactionProcessor();

        System.out.println(
                "=== MINIBANK TRANSACTION PROCESSOR ===");
```

```
        for (int i = 1; i <= 8; i++)
        {
            processor.submit(
                    new TransactionTask(
                            account,
                            100,
                            TransactionTask.Type.DEPOSIT));
        }

        for (int i = 1; i <= 3; i++)
        {
            processor.submit(
                    new TransactionTask(
                            account,
                            50,
                            TransactionTask.Type.WITHDRAW));
        }

        processor.stop();

        System.out.println(
                "\nFinal balance: "
                + account.getBalance());

        System.out.println(
                "Expected balance: 1650");

        System.out.println(
                "Correct: "
                + (account.getBalance() == 1650));
    }
}
```

DeadlockDemo.java

```
import java.lang.management.ManagementFactory;
import java.lang.management.ThreadInfo;
import java.lang.management.ThreadMXBean;
import java.util.concurrent.CountDownLatch;

import model.Account;
import model.SavingsAccount;

public class DeadlockDemo
{
    public static void main(String[] args)
            throws Exception
    {
        // Use separate accounts for the unsafe demonstration.
        // They become intentionally locked, so they must not be
        // reused for the corrected transfer.
        Account unsafeA =
                new SavingsAccount("Unsafe-A", 1000, 0);

        Account unsafeB =
                new SavingsAccount("Unsafe-B", 1000, 0);

        System.out.println(
                "=== DEADLOCK: OPPOSITE LOCK ORDER ===");

        System.out.println(
                "Unsafe A: "
                + unsafeA.getAccountNumber());

        System.out.println(
                "Unsafe B: "
```

```
                + unsafeB.getAccountNumber());

        demonstrateDeadlock(
                unsafeA,
                unsafeB);

        // Fresh accounts for the corrected version.
        Account safeA =
                new SavingsAccount("Safe-A", 1000, 0);

        Account safeB =
                new SavingsAccount("Safe-B", 1000, 0);

        System.out.println(
                "\n=== FIX: SMALLER ACCOUNT NUMBER FIRST ===");

        Thread t1 = new Thread(() ->
        {
            try
            {
                safeTransfer(
                        safeA,
                        safeB,
                        100);
            }
            catch (Exception e)
            {
                System.out.println(
                        "T1 failed: "
                        + e.getMessage());
            }
        }, "Safe-Transfer-A-to-B");

        Thread t2 = new Thread(() ->
        {
            try
            {
                safeTransfer(
                        safeB,
                        safeA,
                        100);
            }
            catch (Exception e)
            {
                System.out.println(
                        "T2 failed: "
                        + e.getMessage());
            }
        }, "Safe-Transfer-B-to-A");

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(
                "\nBoth safe transfers completed.");

        System.out.println(
                safeA.getAccountNumber()
                + " balance = "
                + safeA.getBalance());

        System.out.println(
                safeB.getAccountNumber()
```

```
                + " balance = "
                + safeB.getBalance());
    }

    private static void demonstrateDeadlock(
            Account a,
            Account b)
            throws InterruptedException
    {
        CountDownLatch bothFirstLocks =
                new CountDownLatch(2);

        Thread t1 = new Thread(() ->
        {
            synchronized (a)
            {
                System.out.println(
                        Thread.currentThread().getName()
                        + " locked "
                        + a.getAccountNumber());

                bothFirstLocks.countDown();
                await(bothFirstLocks);

                // Opposite order: a -> b.
                synchronized (b)
                {
                    System.out.println(
                            "T1 acquired both locks.");
                }
            }
        }, "Transfer-A-to-B");

        Thread t2 = new Thread(() ->
        {
            synchronized (b)
            {
                System.out.println(
                        Thread.currentThread().getName()
                        + " locked "
                        + b.getAccountNumber());

                bothFirstLocks.countDown();
                await(bothFirstLocks);

                // Opposite order: b -> a.
                synchronized (a)
                {
                    System.out.println(
                            "T2 acquired both locks.");
                }
            }
        }, "Transfer-B-to-A");

        // Daemon threads make the intentional deadlock harmless.
        t1.setDaemon(true);
        t2.setDaemon(true);

        t1.start();
        t2.start();

        Thread.sleep(500);

        ThreadMXBean bean =
                ManagementFactory.getThreadMXBean();
```

```
        long[] ids =
                bean.findDeadlockedThreads();

        if (ids != null)
        {
            System.out.println(
                    "Deadlock detected: "
                    + ids.length
                    + " threads.");

            ThreadInfo[] info =
                    bean.getThreadInfo(ids);

            for (ThreadInfo thread : info)
            {
                if (thread != null)
                {
                    System.out.println(
                            "Blocked: "
                            + thread.getThreadName());
                }
            }

            System.out.println(
                    "Reason: each thread holds one account "
                    + "lock and waits for the other.");
        }
        else
        {
            System.out.println(
                    "Deadlock was not detected in this run.");
        }
    }

    private static void await(
            CountDownLatch latch)
    {
        try
        {
            latch.await();
        }
        catch (InterruptedException e)
        {
            Thread.currentThread().interrupt();
        }
    }

    /*
     * Correct solution:
     * both directions lock the smaller accountNumber first.
     */
    public static void safeTransfer(
            Account from,
            Account to,
            long amount)
            throws Exception
    {
        Account first;
        Account second;

        if (from.getAccountNumber()
                .compareTo(to.getAccountNumber()) < 0)
        {
            first = from;
            second = to;
        }
```

```
        else
        {
            first = to;
            second = from;
        }

        synchronized (first)
        {
            synchronized (second)
            {
                from.withdraw(amount);
                to.deposit(amount);

                System.out.println(
                        Thread.currentThread().getName()
                        + " transferred "
                        + amount
                        + " | lock order: "
                        + first.getAccountNumber()
                        + " -> "
                        + second.getAccountNumber());
            }
        }
    }
}
```

TransactionBufferDemo.java

```
public class TransactionBufferDemo
{
    private static final int BUFFER_SIZE = 4;
    private static final int TRANSACTION_COUNT = 12;

    public static void main(String[] args)
            throws InterruptedException
    {
        TransactionBuffer buffer =
                new TransactionBuffer(BUFFER_SIZE);

        Thread producer =
                new Thread(
                        new Producer(buffer),
                        "Transaction-Producer");

        Thread consumer =
                new Thread(
                        new Consumer(buffer),
                        "Transaction-Consumer");

        System.out.println(
                "=== TRANSACTION PRODUCER-CONSUMER ===");

        producer.start();
        consumer.start();

        producer.join();
        consumer.join();

        System.out.println(
                "\nProduced: "
                + buffer.getProducedCount());

        System.out.println(
                "Consumed: "
                + buffer.getConsumedCount());
```

```
        System.out.println(
                "Remaining: " + buffer.size());

        boolean correct =
                buffer.getProducedCount()
                        == TRANSACTION_COUNT
                && buffer.getConsumedCount()
                        == TRANSACTION_COUNT
                && buffer.size() == 0;

        System.out.println(
                "Correct result: " + correct);
    }

    private static class Transaction
    {
        private final int id;
        private final long amount;

        Transaction(int id, long amount)
        {
            this.id = id;
            this.amount = amount;
        }

        @Override
        public String toString()
        {
            return "Transaction{id="
                    + id
                    + ", amount="
                    + amount
                    + "}";
        }
    }

    /*
     * Fixed-size ARRAY buffer.
     * head/tail make it a circular FIFO queue.
     */
    private static class TransactionBuffer
    {
        private final Transaction[] buffer;

        private int head;
        private int tail;
        private int count;

        private int produced;
        private int consumed;

        TransactionBuffer(int capacity)
        {
            buffer = new Transaction[capacity];
        }

        public synchronized void put(
                Transaction transaction)
                throws InterruptedException
        {
            while (count == buffer.length)
            {
                System.out.println(
                        "Producer waits: buffer FULL.");
                wait();
            }
```

```
            buffer[tail] = transaction;
            tail = (tail + 1) % buffer.length;
            count++;
            produced++;

            System.out.println(
                    "Produced -> "
                    + transaction
                    + " | bufferCount="
                    + count);

            notify();
        }

        public synchronized Transaction take()
                throws InterruptedException
        {
            while (count == 0)
            {
                System.out.println(
                        "Consumer waits: buffer EMPTY.");
                wait();
            }

            Transaction transaction =
                    buffer[head];

            buffer[head] = null;
            head = (head + 1) % buffer.length;
            count--;
            consumed++;

            System.out.println(
                    "Consumed <- "
                    + transaction
                    + " | bufferCount="
                    + count);

            notify();

            return transaction;
        }

        public synchronized int size()
        {
            return count;
        }

        public synchronized int getProducedCount()
        {
            return produced;
        }

        public synchronized int getConsumedCount()
        {
            return consumed;
        }
    }

    private static class Producer implements Runnable
    {
        private final TransactionBuffer buffer;

        Producer(TransactionBuffer buffer)
        {
```

```
            this.buffer = buffer;
        }

        @Override
        public void run()
        {
            try
            {
                for (int i = 1;
                        i <= TRANSACTION_COUNT;
                        i++)
                {
                    buffer.put(
                            new Transaction(
                                    i,
                                    i * 100L));

                    Thread.sleep(60);
                }
            }
            catch (InterruptedException e)
            {
                Thread.currentThread().interrupt();
            }

            System.out.println(
                    "Transaction producer finished.");
        }
    }

    private static class Consumer implements Runnable
    {
        private final TransactionBuffer buffer;

        Consumer(TransactionBuffer buffer)
        {
            this.buffer = buffer;
        }

        @Override
        public void run()
        {
            try
            {
                for (int i = 1;
                        i <= TRANSACTION_COUNT;
                        i++)
                {
                    buffer.take();
                    Thread.sleep(90);
                }
            }
            catch (InterruptedException e)
            {
                Thread.currentThread().interrupt();
            }

            System.out.println(
                    "Transaction consumer finished.");
        }
    }
}
```

TransactionProcessor.java

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class TransactionProcessor
{
    private final ExecutorService executor;

    public TransactionProcessor()
    {
        // Fixed pool of exactly four worker threads.
        executor =
                Executors.newFixedThreadPool(4);
    }

    public void submit(Runnable transaction)
    {
        executor.execute(transaction);
    }

    public void stop()
            throws InterruptedException
    {
        executor.shutdown();

        if (!executor.awaitTermination(
                30,
                TimeUnit.SECONDS))
        {
            System.out.println(
                    "Timeout: stopping unfinished transactions.");

            executor.shutdownNow();
        }
        else
        {
            System.out.println(
                    "TransactionProcessor stopped cleanly.");
        }
    }
}
```

TransactionTask.java

```
import exception.BankException;
import model.Account;

public class TransactionTask implements Runnable
{
    public enum Type
    {
        DEPOSIT,
        WITHDRAW
    }

    private final Account account;
    private final long amount;
    private final Type type;

    public TransactionTask(
            Account account,
            long amount,
            Type type)
    {
```

```
        this.account = account;
        this.amount = amount;
        this.type = type;
    }

    @Override
    public void run()
    {
        String threadName =
                Thread.currentThread().getName();

        try
        {
            if (type == Type.DEPOSIT)
            {
                account.deposit(amount);

                System.out.println(
                        threadName
                        + " -> DEPOSIT "
                        + amount
                        + " | balance="
                        + account.getBalance());
            }
            else
            {
                account.withdraw(amount);

                System.out.println(
                        threadName
                        + " -> WITHDRAW "
                        + amount
                        + " | balance="
                        + account.getBalance());
            }
        }
        catch (BankException e)
        {
            System.out.println(
                    threadName
                    + " -> transaction failed: "
                    + e.getMessage());
        }
    }
}
```

exception/AccountNotFoundException.java

```
package exception;

public class AccountNotFoundException extends BankException
{
    public AccountNotFoundException(String message)
    {
        super(message);
    }
}
```

exception/BankException.java

```
package exception;

public class BankException extends Exception
{
    public BankException(String message)
    {
```

```
        super(message);
    }
}
```

exception/InsufficientFundsException.java

```
package exception;

public class InsufficientFundsException extends BankException
{
    private final long shortfall;

    public InsufficientFundsException(String message, long shortfall)
    {
        super(message);
        this.shortfall = shortfall;
    }

    public long getShortfall()
    {
        return shortfall;
    }
}
```

exception/InvalidAmountException.java

```
package exception;

public class InvalidAmountException extends BankException
{
    public InvalidAmountException(String message)
    {
        super(message);
    }
}
```

model/Account.java

```
package model;

import exception.BankException;
import exception.InvalidAmountException;
import exception.InsufficientFundsException;
import model.annotation.Id;
import model.annotation.MaxLength;
import model.annotation.Positive;

public abstract class Account
        implements Transactable, InterestBearing
{
    @Id
    private final String accountNumber;

    @MaxLength(30)
    private String ownerName;

    @Positive
    protected long balance;

    private boolean active;

    private static int accountCounter = 1;

    private static String generateAccountNumber()
    {
        return String.format("AC%04d", accountCounter++);
```

```
    }

    public Account(String ownerName, long openingBalance)
    {
        this.accountNumber = generateAccountNumber();
        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }

    public Account(String ownerName)
    {
        this(ownerName, 0);
    }

    public String getAccountNumber()
    {
        return accountNumber;
    }

    public String getOwnerName()
    {
        return ownerName;
    }

    public long getBalance()
    {
        return balance;
    }

    public boolean isActive()
    {
        return active;
    }

    @Override
    public synchronized void deposit(long amount) throws InvalidAmountException
    {
        if (amount <= 0)
        {
            throw new InvalidAmountException(
                    "Deposit amount must be greater than zero.");
        }

        balance += amount;
    }

    @Override
    public synchronized boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException
    {
        if (amount <= 0)
        {
            throw new InvalidAmountException(
                    "Withdrawal amount must be greater than zero.");
        }

        if (!canWithdraw(amount))
        {
            long available = Math.max(0, balance);
            long shortfall = amount - available;

            throw new InsufficientFundsException(
                    "Insufficient funds. Balance: " + balance
                    + ", requested: " + amount,
                    Math.max(0, shortfall));
```

```
        }

        balance -= amount;
        return true;
    }

    public void transfer(Account to, long amount) throws BankException
    {
        try
        {
            if (to == null)
            {
                throw new exception.AccountNotFoundException(
                        "Destination account was not found.");
            }

            withdraw(amount);
            to.deposit(amount);

            System.out.println(
                    "Transfer successful: " + amount
                    + " from " + getAccountNumber()
                    + " to " + to.getAccountNumber());
        }
        catch (BankException e)
        {
            System.out.println(
                    "Transfer failed: " + e.getMessage());
            throw e;
        }
        finally
        {
            System.out.println(
                    "Transfer operation finished for "
                    + getAccountNumber());
        }
    }

    public abstract double interestRate();

    public abstract boolean canWithdraw(long amount);

    @Override
    public double balanceForInterest()
    {
        return balance;
    }

    @Override
    public String toString()
    {
        return accountNumber + " | "
                + ownerName + " | "
                + balance + " | active=" + active;
    }

    @Override
    public boolean equals(Object obj)
    {
        if (this == obj)
        {
            return true;
        }

        if (!(obj instanceof Account))
        {
```

```
            return false;
        }

        Account other = (Account) obj;

        return accountNumber.equals(other.accountNumber);
    }

    @Override
    public int hashCode()
    {
        return accountNumber.hashCode();
    }
}
```

model/annotation/Id.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Id
{
}
```

model/annotation/MaxLength.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface MaxLength
{
    int value();
}
```

model/annotation/Positive.java

```
package model.annotation;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Positive
{
    String message() default "must be > 0";
}
```

model/BankInfo.java

```
package model;
```

```
public record BankInfo(String name, String branch)
{
}
```

model/Command.java

```
package model;

public record Command(
        TransactionType type,
        String accountNumber,
        long amount)
{
}
```

model/CurrentAccount.java

```
package model;

public class CurrentAccount extends Account
{
    private long overdraftLimit;

    public CurrentAccount(
            String ownerName,
            long balance,
            long overdraftLimit)
    {
        super(ownerName, balance);
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public double interestRate()
    {
        return 0.0;
    }

    @Override
    public boolean canWithdraw(long amount)
    {
        return amount >= 0
                && balance - amount >= -overdraftLimit;
    }
}
```

model/Customer.java

```
package model;

public class Customer implements Cloneable
{
    private String name;
    private String email;
    private String mobile;
    private final String customerId;

    private static long customerCounter = 100;

    private Address address;

    private static String generateCustomerId()
    {
        return "CUST" + (++customerCounter);
    }
```

```
    public Customer(String name, String email, String mobile)
    {
        this.customerId = generateCustomerId();
        this.name = name;
        this.email = email;
        this.mobile = mobile;
    }

    public String getName()
    {
        return name;
    }

    public String getEmail()
    {
        return email;
    }

    public String getMobile()
    {
        return mobile;
    }

    public String getCustomerId()
    {
        return customerId;
    }

    public Address getAddress()
    {
        return address;
    }

    public void setAddress(Address address)
    {
        this.address = address;
    }

    @Override
    public Customer clone()
    {
        try
        {
            Customer copy = (Customer) super.clone();

            if (address != null)
            {
                copy.address = new Address(
                        address.getLine(),
                        address.getCity(),
                        address.getPincode()
                );
            }

            return copy;
        }
        catch (CloneNotSupportedException e)
        {
            return null;
        }
    }

    public static class Address
    {
        private String line;
        private String city;
```

```
        private String pincode;

        public Address(String line, String city, String pincode)
        {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }

        public String getLine()
        {
            return line;
        }

        public String getCity()
        {
            return city;
        }

        public String getPincode()
        {
            return pincode;
        }

        @Override
        public String toString()
        {
            return line + ", " + city + " - " + pincode;
        }
    }
}
```

model/FixedDepositAccount.java

```
package model;

public class FixedDepositAccount extends Account
{
    public FixedDepositAccount(String ownerName, long balance)
    {
        super(ownerName, balance);
    }

    @Override
    public double interestRate()
    {
        return 7.0;
    }

    @Override
    public boolean canWithdraw(long amount)
    {
        return false;
    }
}
```

model/InterestBearing.java

```
package model;

public interface InterestBearing
{
    default double yearlyInterest()
    {
        return balanceForInterest() * interestRate() / 100.0;
    }
```

```
    double interestRate();

    double balanceForInterest();
}
```

model/MenuOption.java

```
package model;

public enum MenuOption
{
    OPEN_ACCOUNT,
    DEPOSIT,
    WITHDRAW,
    TRANSFER,
    EXIT
}
```

model/Premium.java

```
package model;

public interface Premium
{
}
```

model/RaceAccount.java

```
package model;

import exception.InsufficientFundsException;
import exception.InvalidAmountException;

public class RaceAccount extends Account
{
    public RaceAccount(String ownerName, long balance)
    {
        super(ownerName, balance);
    }

    // Deliberately UNSYNCHRONIZED for the first race-condition demo.
    @Override
    public void deposit(long amount)
            throws InvalidAmountException
    {
        if (amount <= 0)
        {
            throw new InvalidAmountException(
                    "Deposit amount must be greater than zero.");
        }

        long oldBalance = balance;
        Thread.yield();
        balance = oldBalance + amount;
    }

    @Override
    public synchronized boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException
    {
        if (amount <= 0)
        {
            throw new InvalidAmountException(
                    "Withdrawal amount must be greater than zero.");
        }
```

```
        if (!canWithdraw(amount))
        {
            long shortfall =
                    amount - Math.max(0, balance);

            throw new InsufficientFundsException(
                    "Insufficient funds.",
                    Math.max(0, shortfall));
        }

        balance -= amount;
        return true;
    }

    @Override
    public double interestRate()
    {
        return 0.0;
    }

    @Override
    public boolean canWithdraw(long amount)
    {
        return amount >= 0 && balance >= amount;
    }
}
```

model/SavingsAccount.java

```
package model;

public class SavingsAccount extends Account implements Premium
{
    private long minBalance;

    public SavingsAccount(String ownerName, long balance, long minBalance)
    {
        super(ownerName, balance);
        this.minBalance = minBalance;
    }

    public SavingsAccount(String ownerName)
    {
        this(ownerName, 0, 1000);
    }

    @Override
    public double interestRate()
    {
        return 4.0;
    }

    @Override
    public boolean canWithdraw(long amount)
    {
        return amount >= 0 && balance - amount >= minBalance;
    }
}
```

model/Transactable.java

```
package model;

import exception.InvalidAmountException;
import exception.InsufficientFundsException;
```

```
public interface Transactable
{
    void deposit(long amount) throws InvalidAmountException;

    boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException;
}
```

model/TransactionType.java

```
package model;

public enum TransactionType
{
    DEPOSIT,
    WITHDRAW,
    TRANSFER
}
```

model/WithdrawRule.java

```
package model;

@FunctionalInterface
public interface WithdrawRule
{
    boolean allow(Account account, long amount);
}
```

service/BankService.java

```
package service;

import exception.BankException;
import model.Account;
import util.StatementFormatter;

public class BankService
{
    public static boolean transfer(
            Account from,
            Account to,
            long amount) throws BankException
    {
        from.transfer(to, amount);
        return true;
    }

    public static String buildStatement(Account account)
    {
        return StatementFormatter.buildStatement(account);
    }
}
```

util/AnnotationValidator.java

```
package util;

import model.annotation.MaxLength;
import model.annotation.Positive;

import java.lang.reflect.Field;
import java.util.ArrayList;

public class AnnotationValidator
```

```
 {
     public static String[] validate(Object obj)
     {
         ArrayList<String> errors = new ArrayList<>();

         if (obj == null)
         {
             errors.add("Object cannot be null");
             return errors.toArray(new String[0]);
         }

         Field[] fields = obj.getClass().getDeclaredFields();

         for (Field field : fields)
         {
             field.setAccessible(true);

             try
             {
                 if (field.isAnnotationPresent(Positive.class))
                 {
                     Positive positive =
                             field.getAnnotation(Positive.class);

                     Object value = field.get(obj);

                     if (value instanceof Number)
                     {
                         long number =
                                 ((Number) value).longValue();

                         if (number <= 0)
                         {
                             errors.add(positive.message());
                         }
                     }
                 }

                 if (field.isAnnotationPresent(MaxLength.class))
                 {
                     MaxLength maxLength =
                             field.getAnnotation(MaxLength.class);

                     Object value = field.get(obj);

                     if (value instanceof String)
                     {
                         String text = (String) value;

                         if (text.length() > maxLength.value())
                         {
                             errors.add(
                                     field.getName()
                                     + " length must be <= "
                                     + maxLength.value()
                             );
                         }
                     }
                 }
             }
             catch (IllegalAccessException e)
             {
                 errors.add("Cannot access field: "
                         + field.getName());
             }
         }
```

```
        return errors.toArray(new String[0]);
    }
}
```

util/CommandParser.java

```
package util;

import model.Command;
import model.TransactionType;

public class CommandParser
{
    public static Command parse(String line)
    {
        if (line == null)
        {
            return null;
        }

        String[] parts = line.trim().split("\\s+");

        if (parts.length != 3)
        {
            return null;
        }

        try
        {
            TransactionType type =
                    TransactionType.valueOf(parts[0].toUpperCase());

            long amount = Long.parseLong(parts[2]);

            if (amount <= 0)
            {
                return null;
            }

            return new Command(type, parts[1], amount);
        }
        catch (IllegalArgumentException e)
        {
            return null;
        }
    }
}
```

util/StatementFormatter.java

```
package util;

import model.Account;

public class StatementFormatter
{
    public static String buildStatement(Account account)
    {
        StringBuilder statement = new StringBuilder();

        statement.append("----- ACCOUNT STATEMENT -----\n");
        statement.append("Account Number: ")
                .append(account.getAccountNumber())
                .append("\n");
        statement.append("Owner: ")
```

```
                .append(account.getOwnerName())
                .append("\n");
        statement.append("Balance: ")
                .append(account.getBalance())
                .append("\n");
        statement.append("Active: ")
                .append(account.isActive())
                .append("\n");
        statement.append("----------------------------");

        return statement.toString();
    }
}
```

util/Validator.java

```
package util;

import java.util.regex.Pattern;

public class Validator
{
    private static final Pattern MOBILE =
            Pattern.compile("^[6-9][0-9]{9}$");

    private static final Pattern EMAIL =
            Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");

    private static final Pattern PAN =
            Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");

    private static final Pattern IFSC =
            Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

    public static boolean isValidMobile(String value)
    {
        return value != null && MOBILE.matcher(value).matches();
    }

    public static boolean isValidEmail(String value)
    {
        return value != null && EMAIL.matcher(value).matches();
    }

    public static boolean isValidPan(String value)
    {
        return value != null && PAN.matcher(value).matches();
    }

    public static boolean isValidIfsc(String value)
    {
        return value != null && IFSC.matcher(value).matches();
    }

    public static boolean isValidAmount(String value)
    {
        return value != null && value.matches("^[1-9][0-9]*$");
    }
}
```

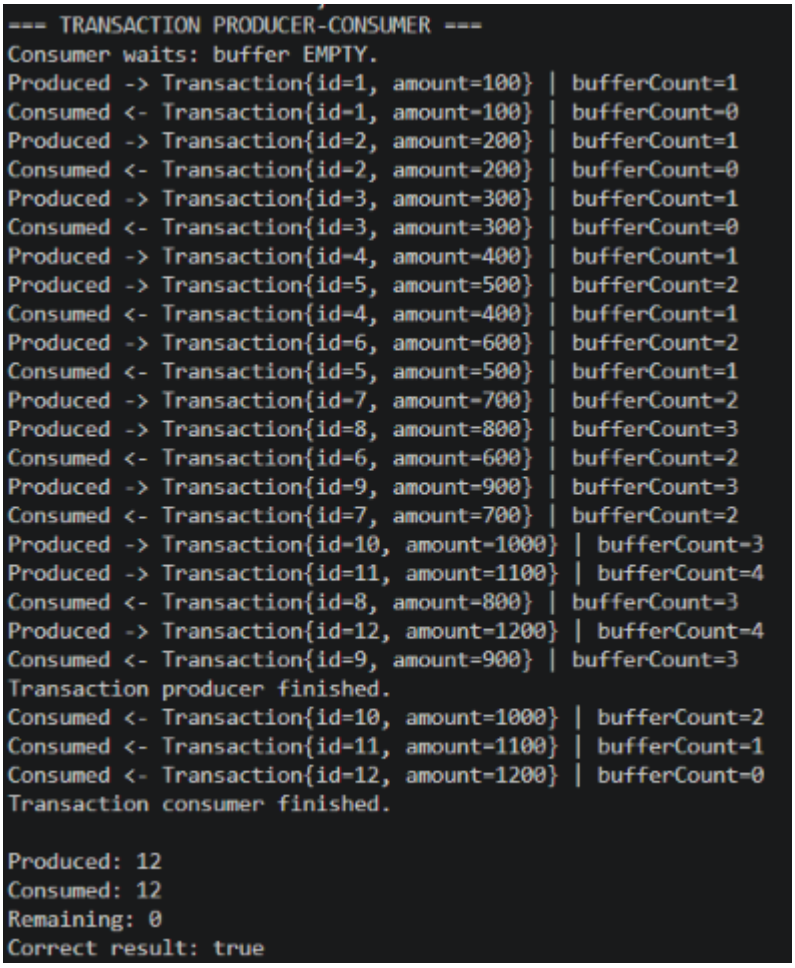
```
=== MINIBANK TRANSACTION PROCESSOR ===
pool-1-thread-2 -> DEPOSIT 100 | balance=1200
pool-1-thread-4 -> DEPOSIT 100 | balance=1400
pool-1-thread-2 -> DEPOSIT 100 | balance=1500
pool-1-thread-4 -> DEPOSIT 100 | balance=1600
pool-1-thread-2 -> DEPOSIT 100 | balance=1700
pool-1-thread-3 -> DEPOSIT 100 | balance=1300
pool-1-thread-2 -> WITHDRAW 50 | balance=1750
pool-1-thread-4 -> DEPOSIT 100 | balance=1800
pool-1-thread-1 -> DEPOSIT 100 | balance=1100
pool-1-thread-3 -> WITHDRAW 50 | balance=1700
pool-1-thread-2 -> WITHDRAW 50 | balance=1650
TransactionProcessor stopped cleanly.

Final balance: 1650
Expected balance: 1650
Correct: true
```

```
=== DEADLOCK: OPPOSITE LOCK ORDER ===
Unsafe A: AC0001
Unsafe B: AC0002
Transfer-B-to-A locked AC0002
Transfer-A-to-B locked AC0001
Deadlock detected: 2 threads.
Blocked: Transfer-A-to-B
Blocked: Transfer-B-to-A
Reason: each thread holds one account lock and waits for the other.

=== FIX: SMALLER ACCOUNT NUMBER FIRST ===
Safe-Transfer-A-to-B transferred 100 | lock order: AC0003 -> AC0004
Safe-Transfer-B-to-A transferred 100 | lock order: AC0003 -> AC0004

Both safe transfers completed.
AC0003 balance = 1000
AC0004 balance = 1000
```



KEY QUESTIONS AND ANSWERS

Q1. Why use a thread pool instead of creating a new thread for every task?

Answer: A thread pool reuses a bounded number of worker threads, reducing creation overhead and preventing too many threads from competing for system resources.

Q2. How do wait() and notifyAll() coordinate producer and consumer threads?

Answer: wait() releases the monitor and pauses a thread until the shared buffer changes. notifyAll() wakes waiting threads after an item is added or removed so they can recheck the buffer condition.

Q3. What is deadlock, and how does consistent lock ordering prevent it?

Answer: Deadlock happens when threads hold locks while each waits for a lock held by another. Locking accounts in the same order, such as lower account number first, removes the circular wait.